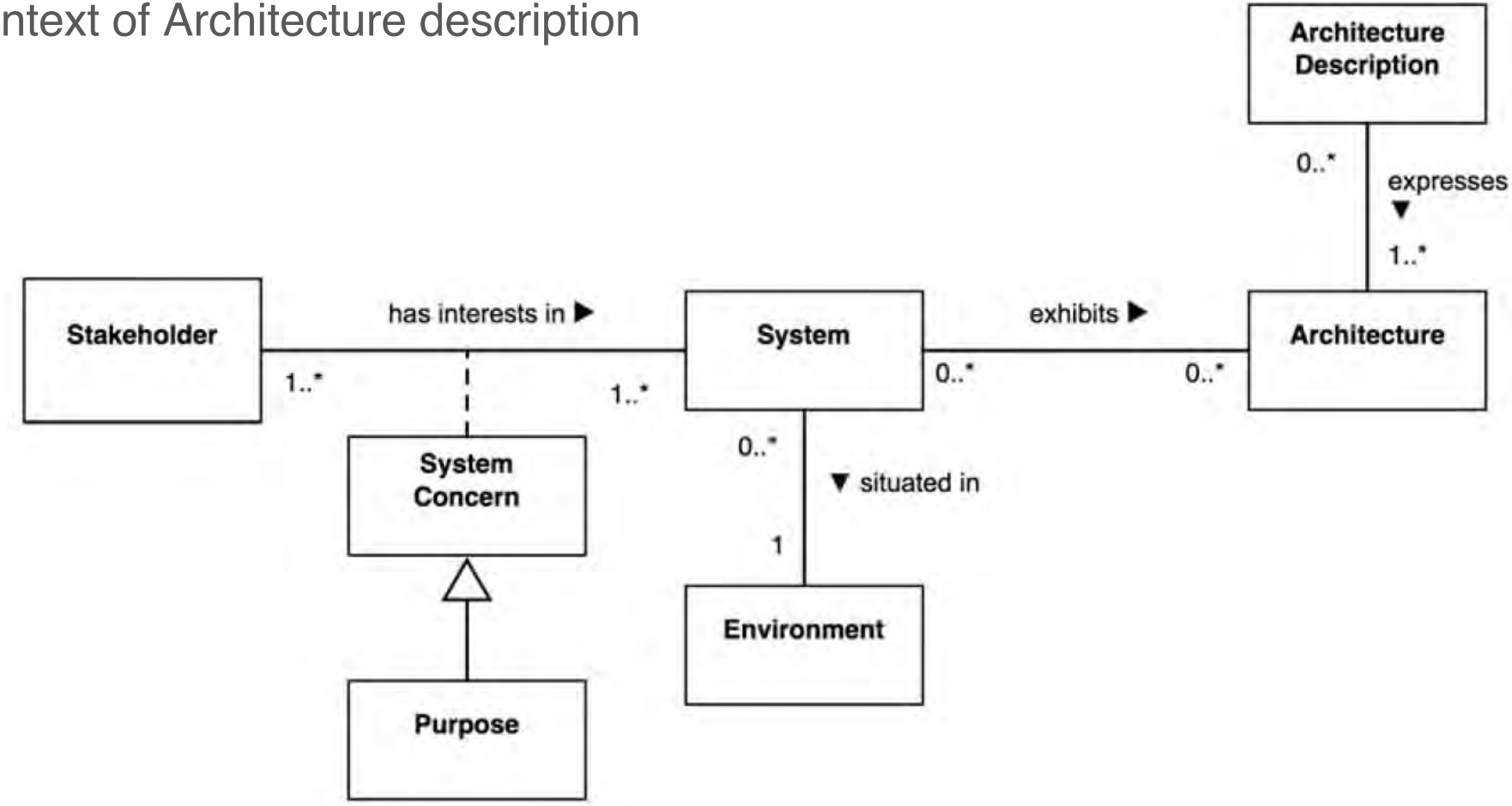
Τεχνολογίες Υπηρεσιών Λογισμικού

Service-Oriented Architectures

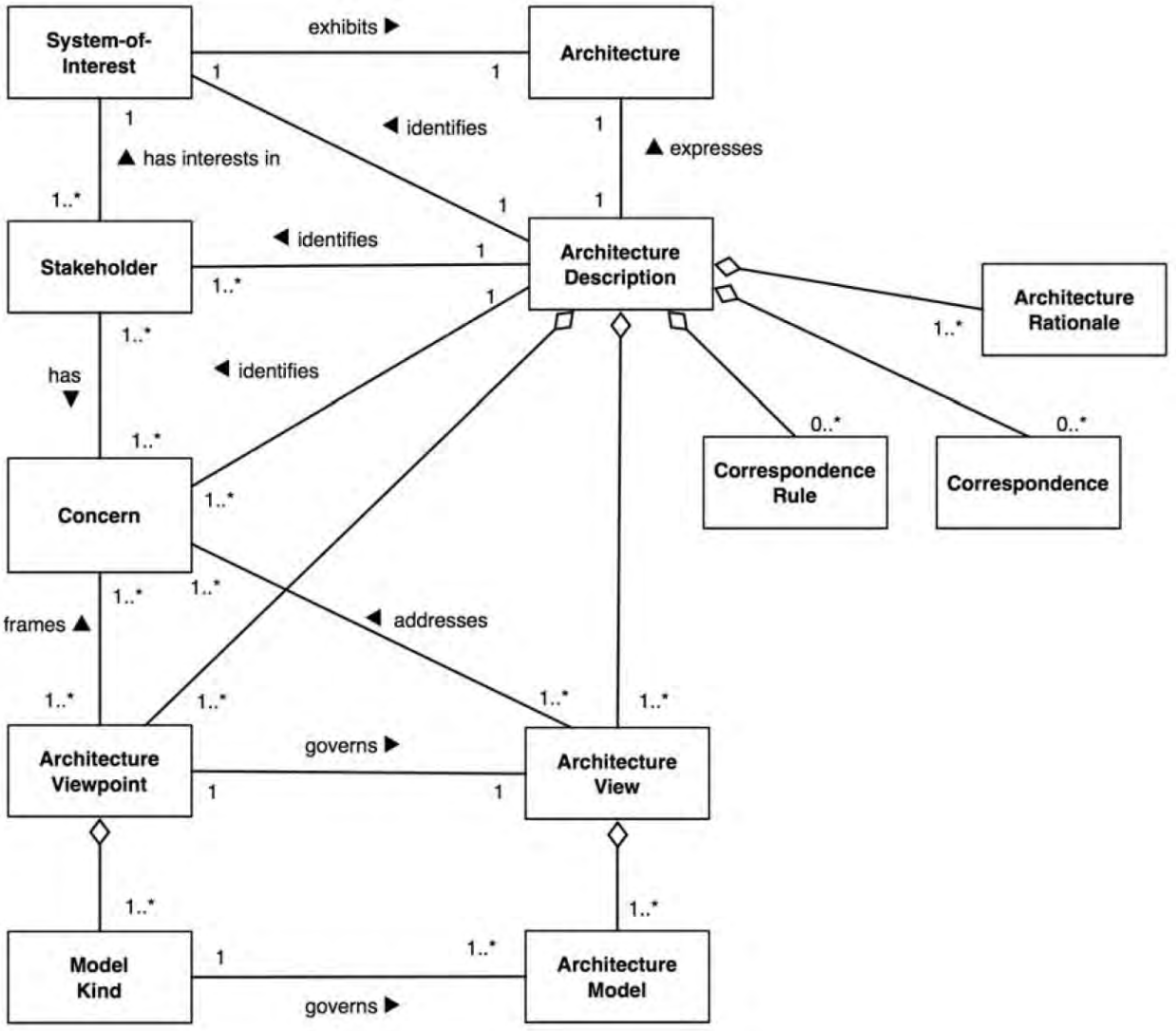
ISO/IEC/IEEE 42010:2011

Context of Architecture description



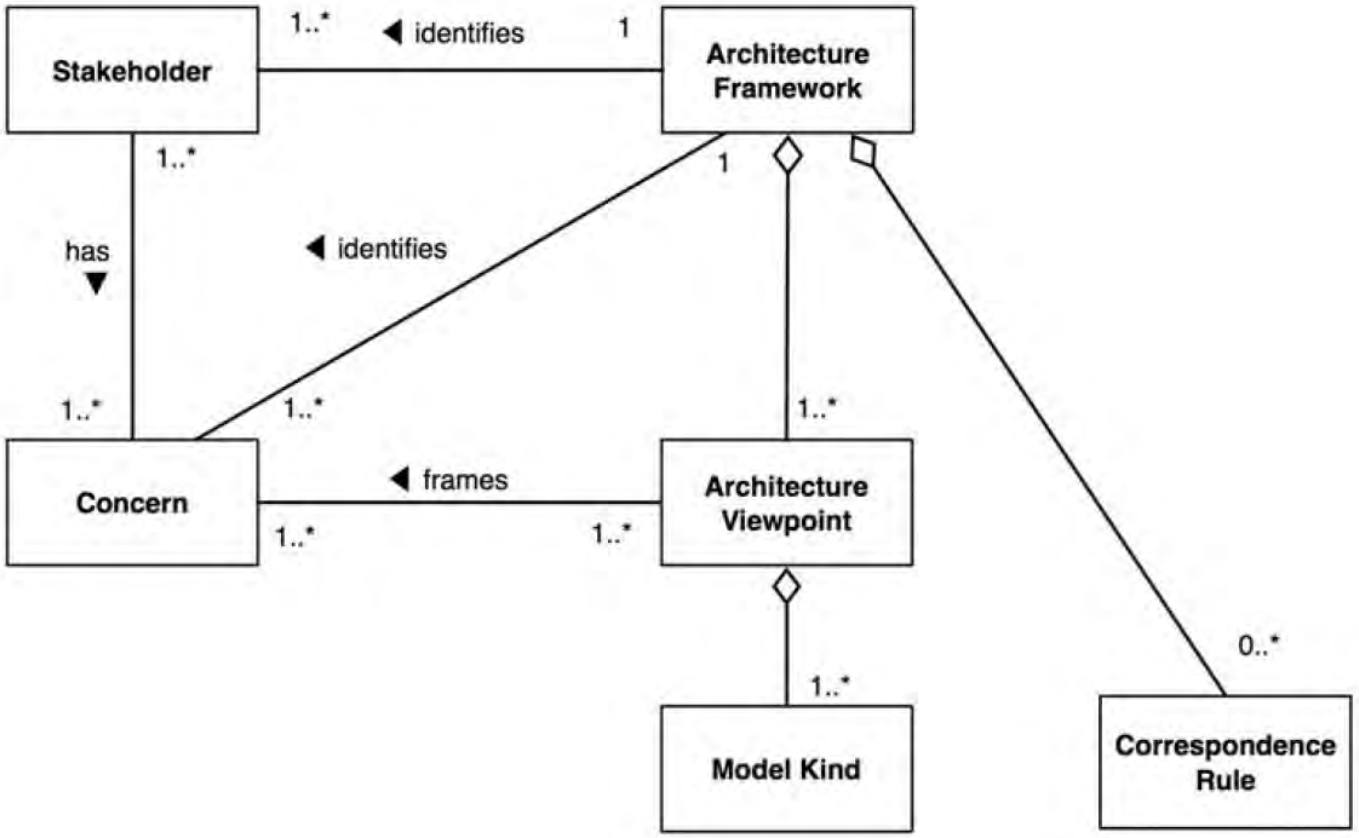
ISO/IEC/IEEE 42010:2011

Conceptual model
of Architecture description



ISO/IEC/IEEE 42010:2011

Conceptual model of Architecture framework



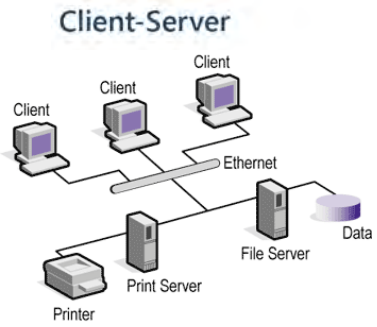
Software architecture

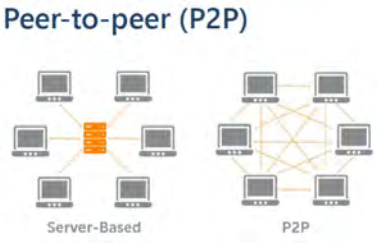
Fundamental structural and design decisions about software, which, once taken, is expensive (requires way too much effort) to change.

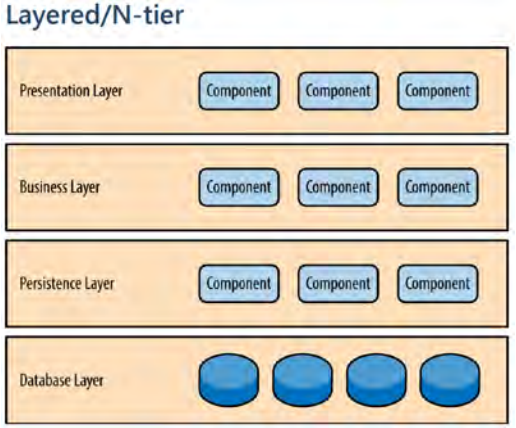
- What are the elements of a software system (structure)
- How are they connected to each other (interfaces)
- How are functional and non-functional requirements assigned to elements and interfaces (responsibilities)

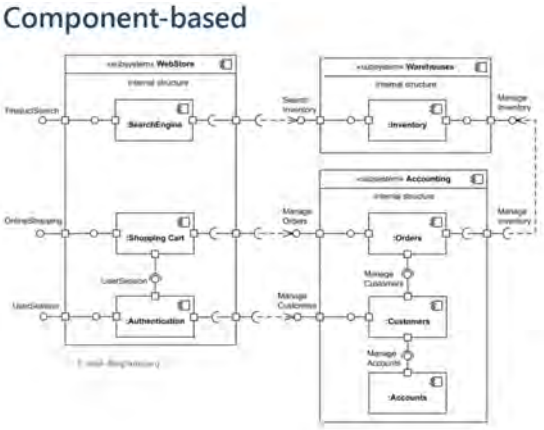
Software architecture

- What are the elements of a software system (structure)
- How are they connected to each other (interfaces)
- How are functional and non-functional requirements assigned to elements and interfaces (responsibilities)







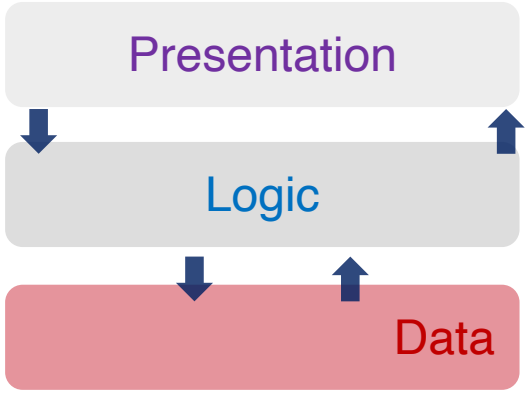


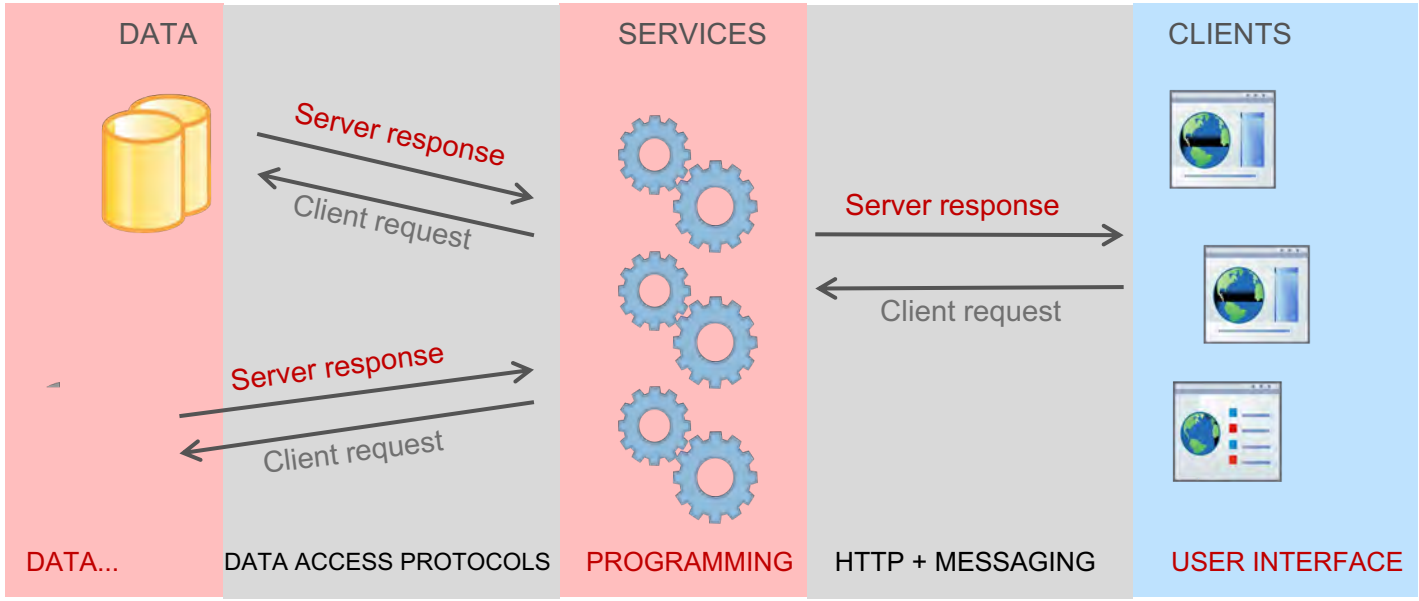
3-tier: data, processing, presentation

Data: What data is needed

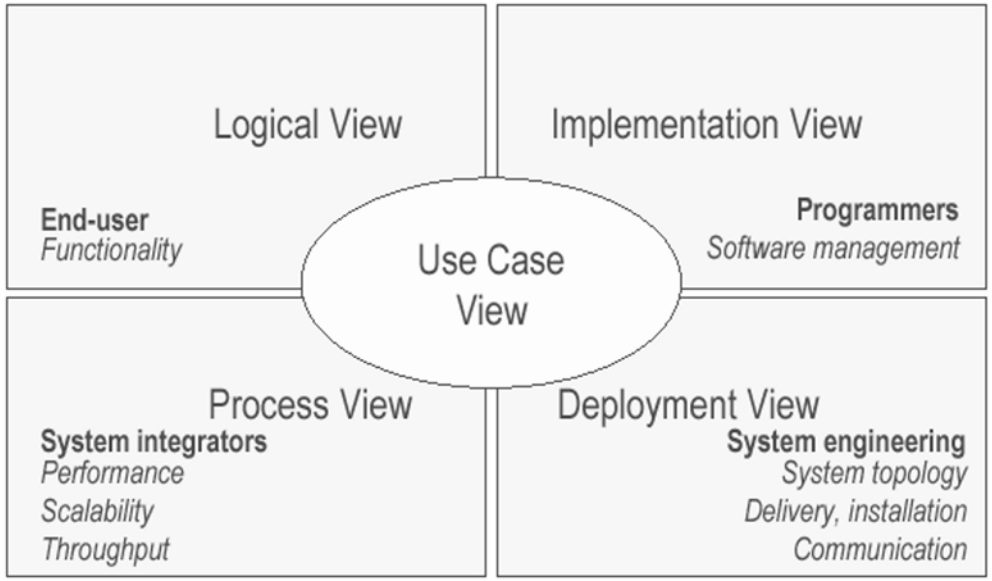
Logic: What to do with data

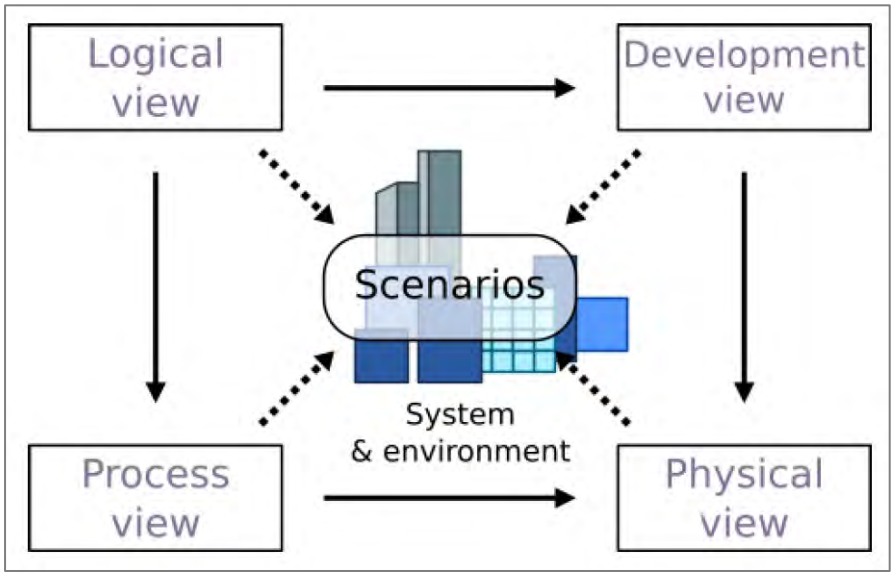
Presentation: Web, desktop, mobile





Architectural views





Service-Oriented Architecture

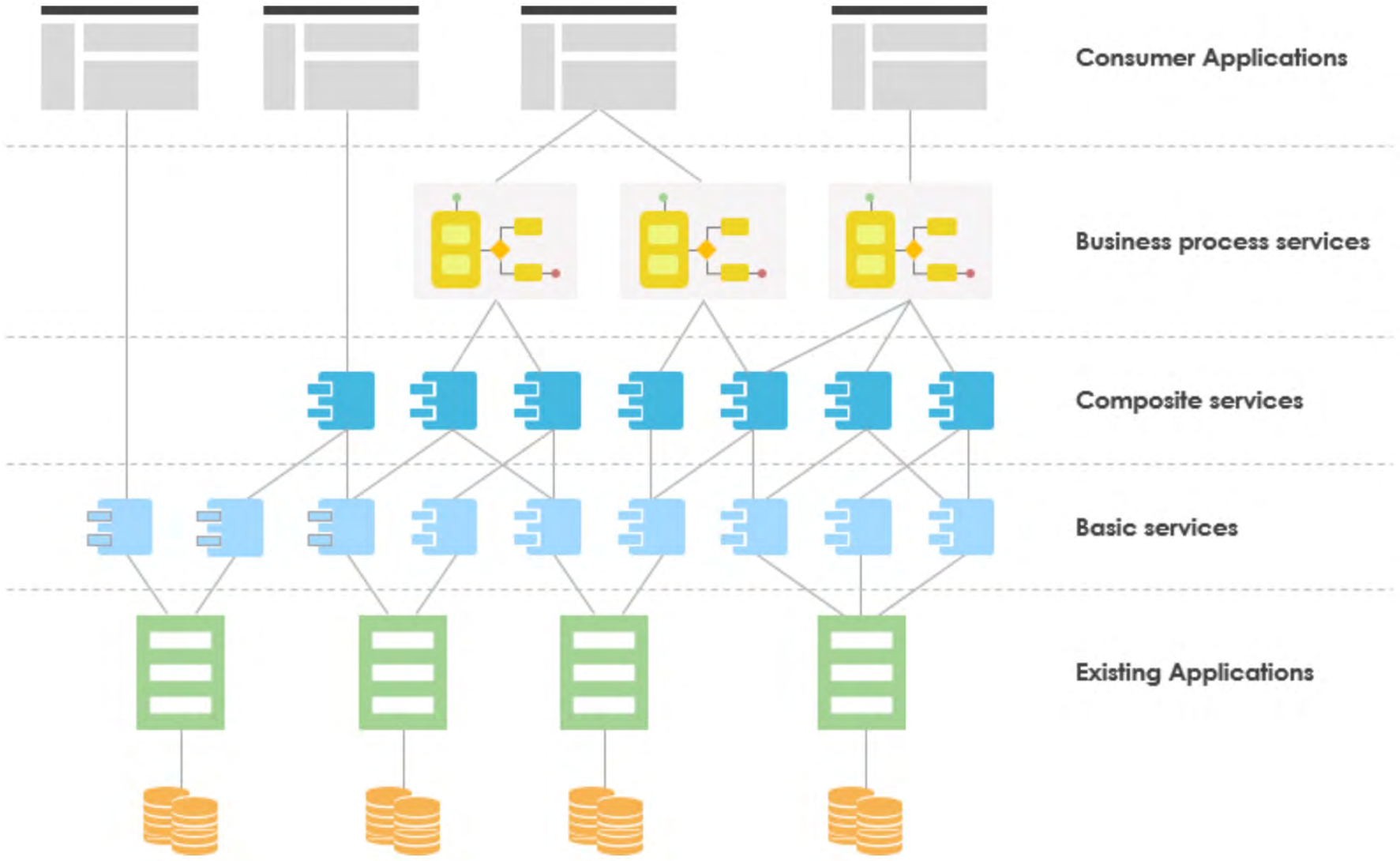
An architectural pattern

- Application components or even full applications provide services to each other
- Services are offered over a network
- Services are independent of technology, vendor, etc.

SOA services...

- Are independent from each other
- Exchange data
- Do not need human interaction

SOA: Service-Oriented Architecture



Source: visual-paradigm.com

Architectural patterns & styles

Architectural style:

- a set of attributes that define some "identity"

Architectural pattern:

- a set of methods to implement a specific style

Not easy to differentiate styles from patterns

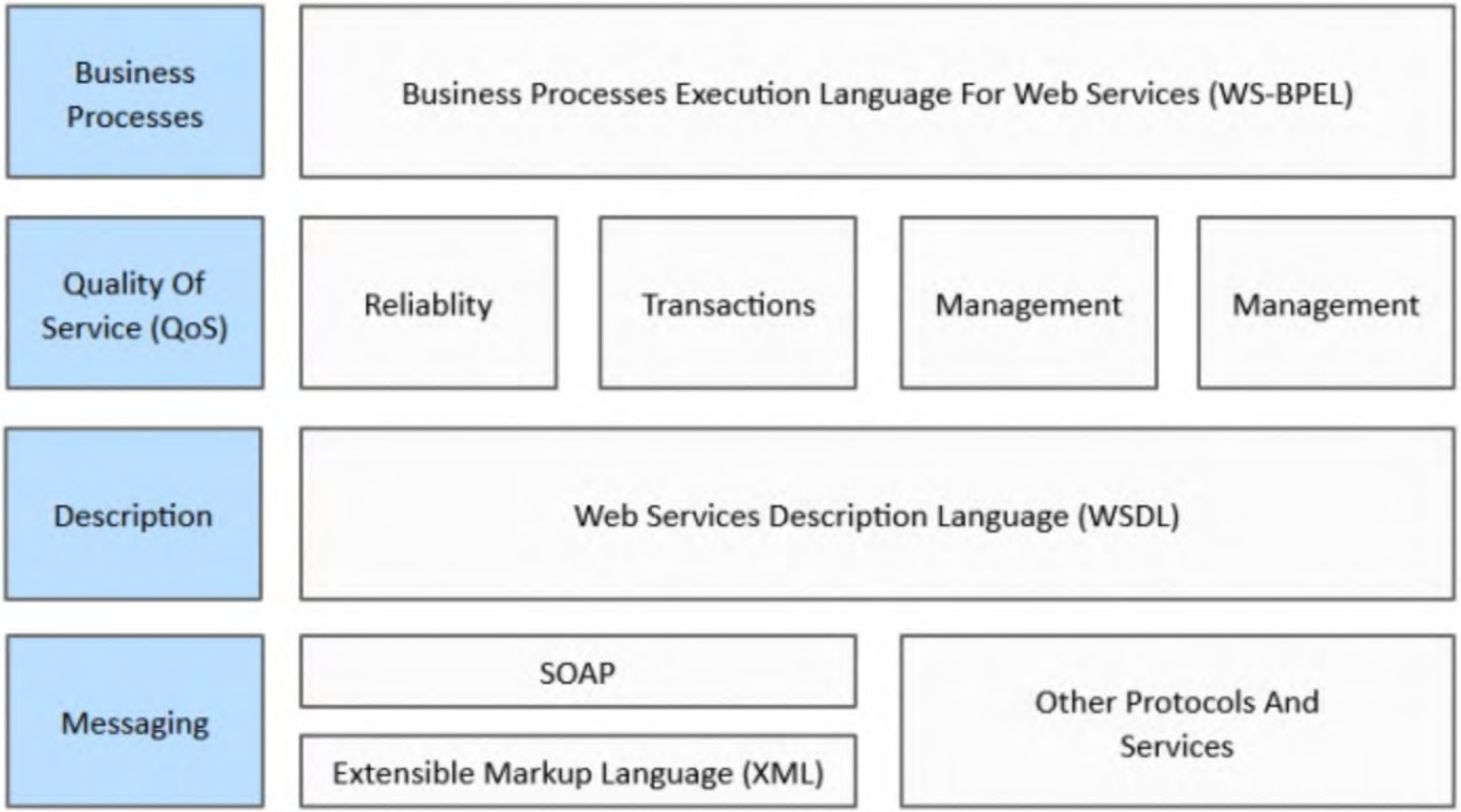
- Specific development / operation environments
- Definition of possible subsystems
- Roles/responsibilities/attributes to subsystems

Service-Oriented Architecture

Why SOA?

- Design components "loosely-coupled" to applications, in a way that can be re-used by other components and applications
- Offer a mechanism to publish services, so that they can be discovered by any existing or future service, regardless of the service owner(s)
- Manage the use of services to avoid security and governance problems

Service-Oriented Architecure



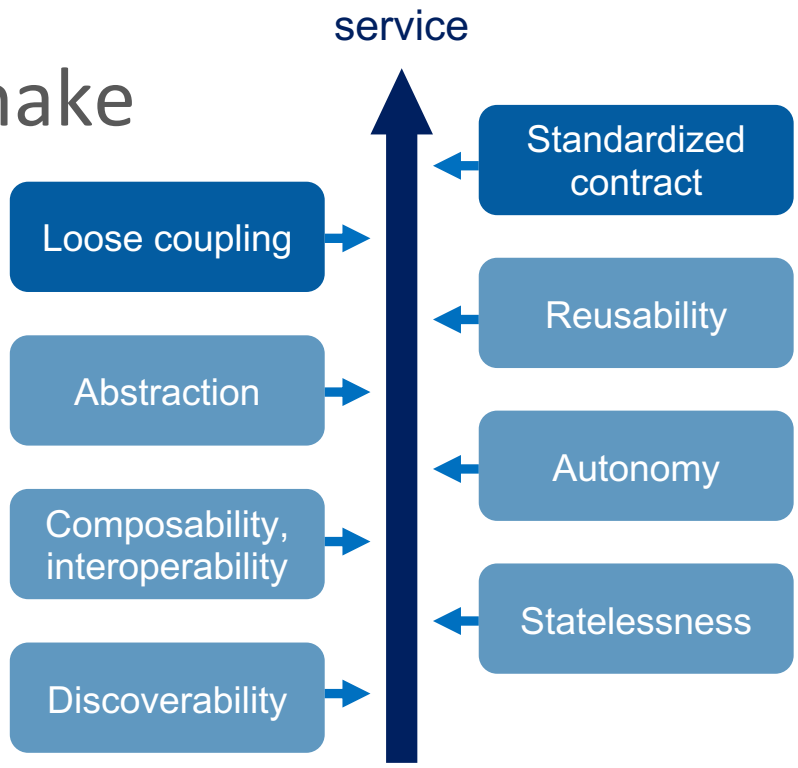
SOA principles

Standardized Service Contract

- A service has a description to make it usable by client applications

Loose coupling

- Services do not depend on each other
- Clients not to be affected by changes in the functionality of services



SOA principles

Abstraction

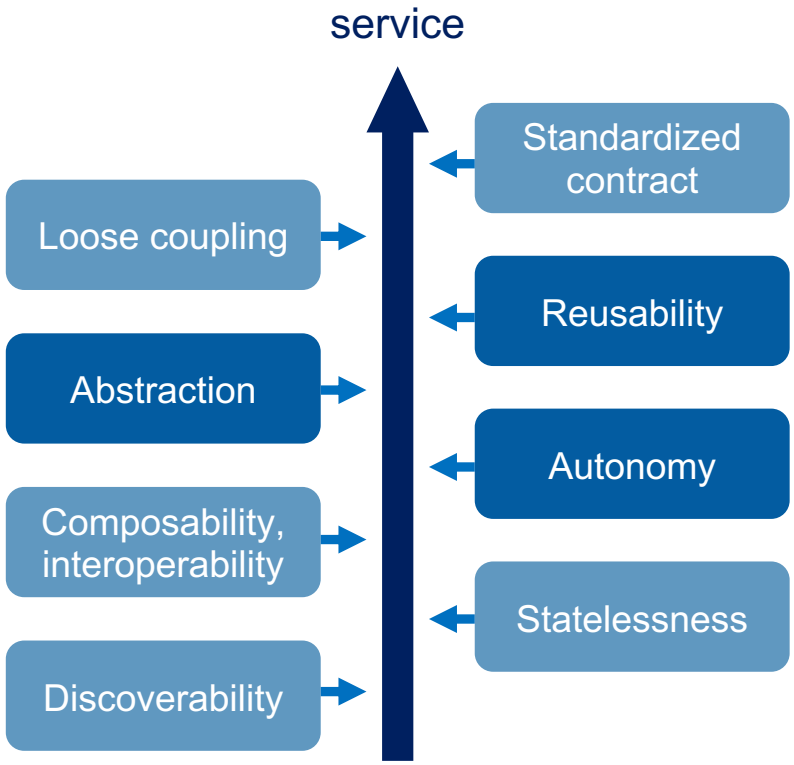
- Service logic should be hidden from the outside

Reusability

- Division of logic should allow re-usability by other apps

Autonomy

- A service "knows" what is needed to offer its functionality



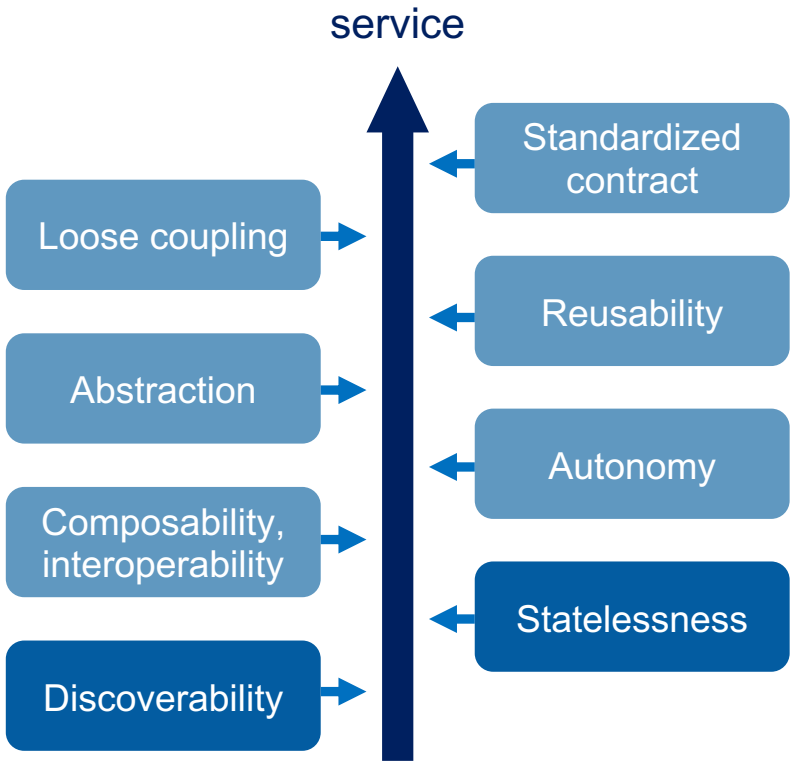
SOA principles

Statelessness

- The service itself should not maintain states
- The client should know what is needed to switch between states

Discoverability

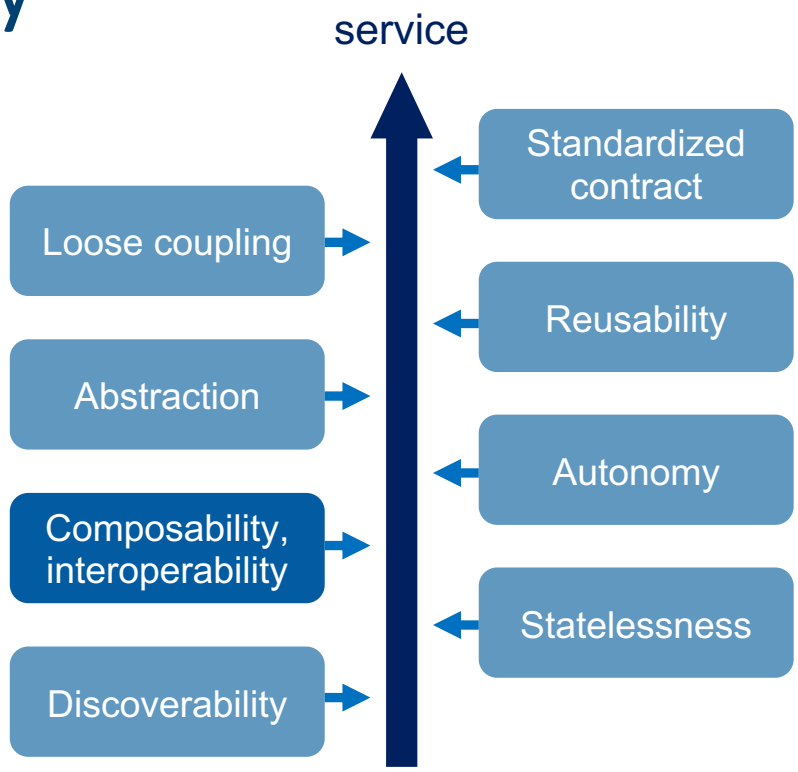
- Services can be discovered in some service registry



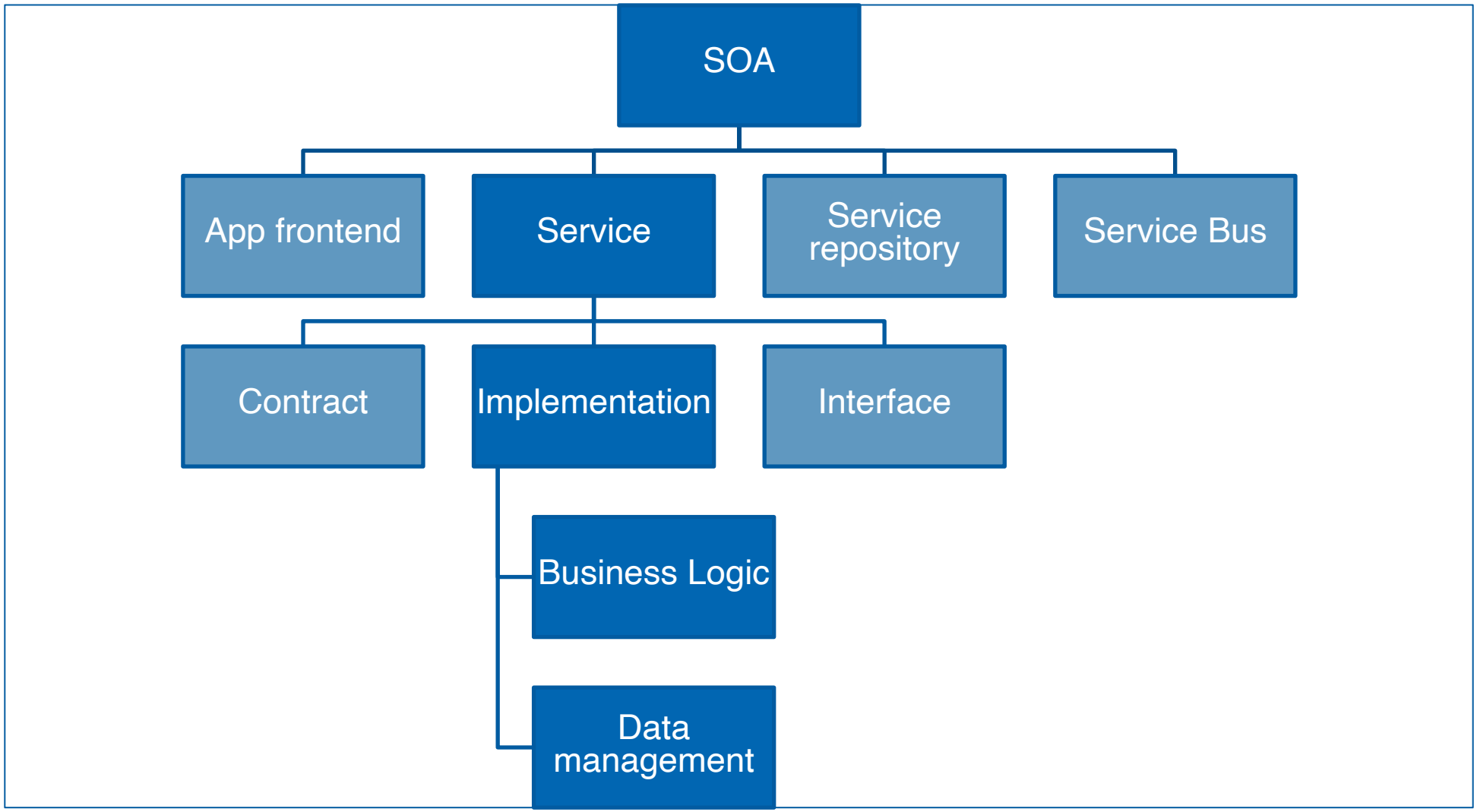
SOA principles

Composability, interoperability

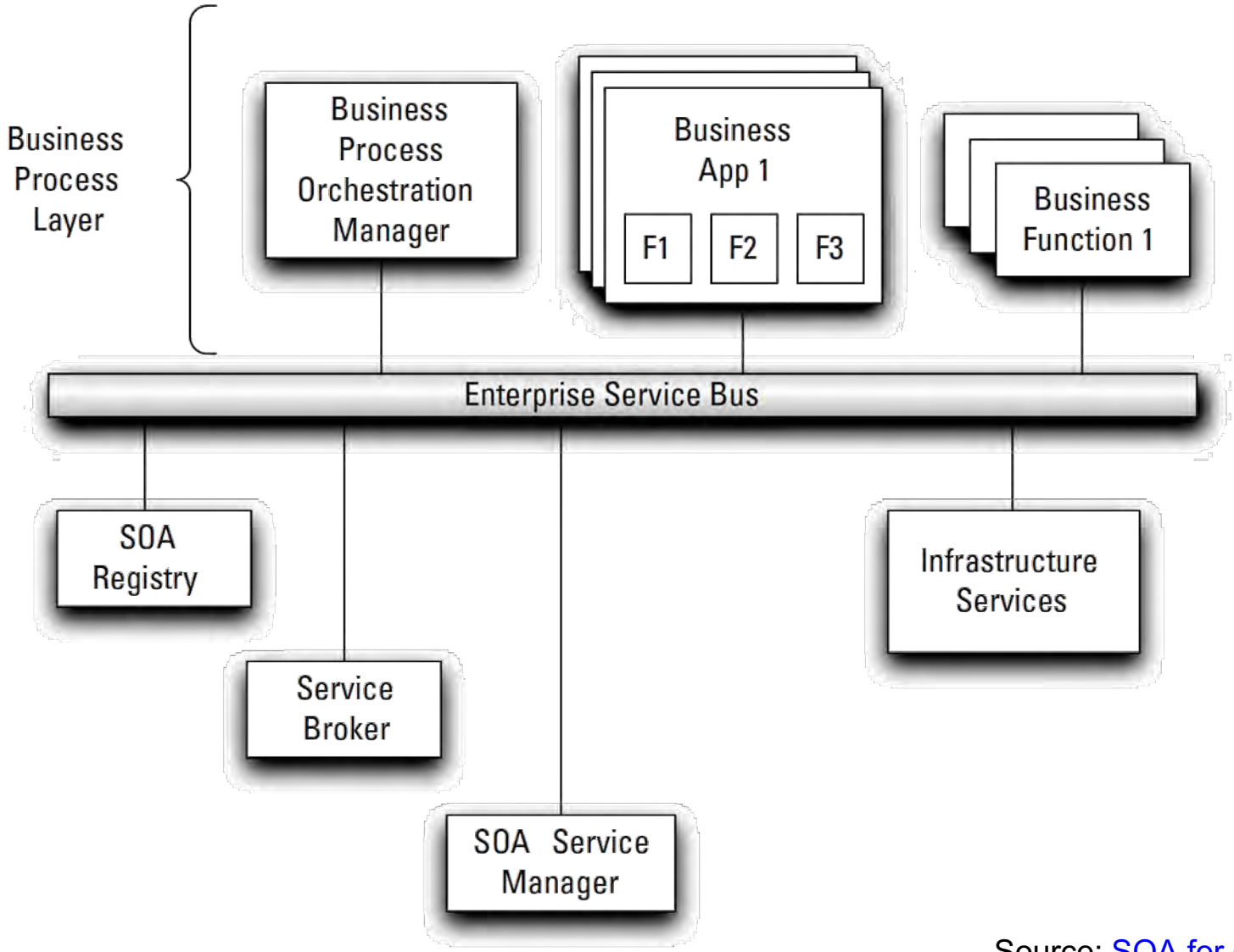
- Business functionality broken down into services
- Use XML, JSON etc. to allow diverse subscribers to use services



Service-Oriented Architecure

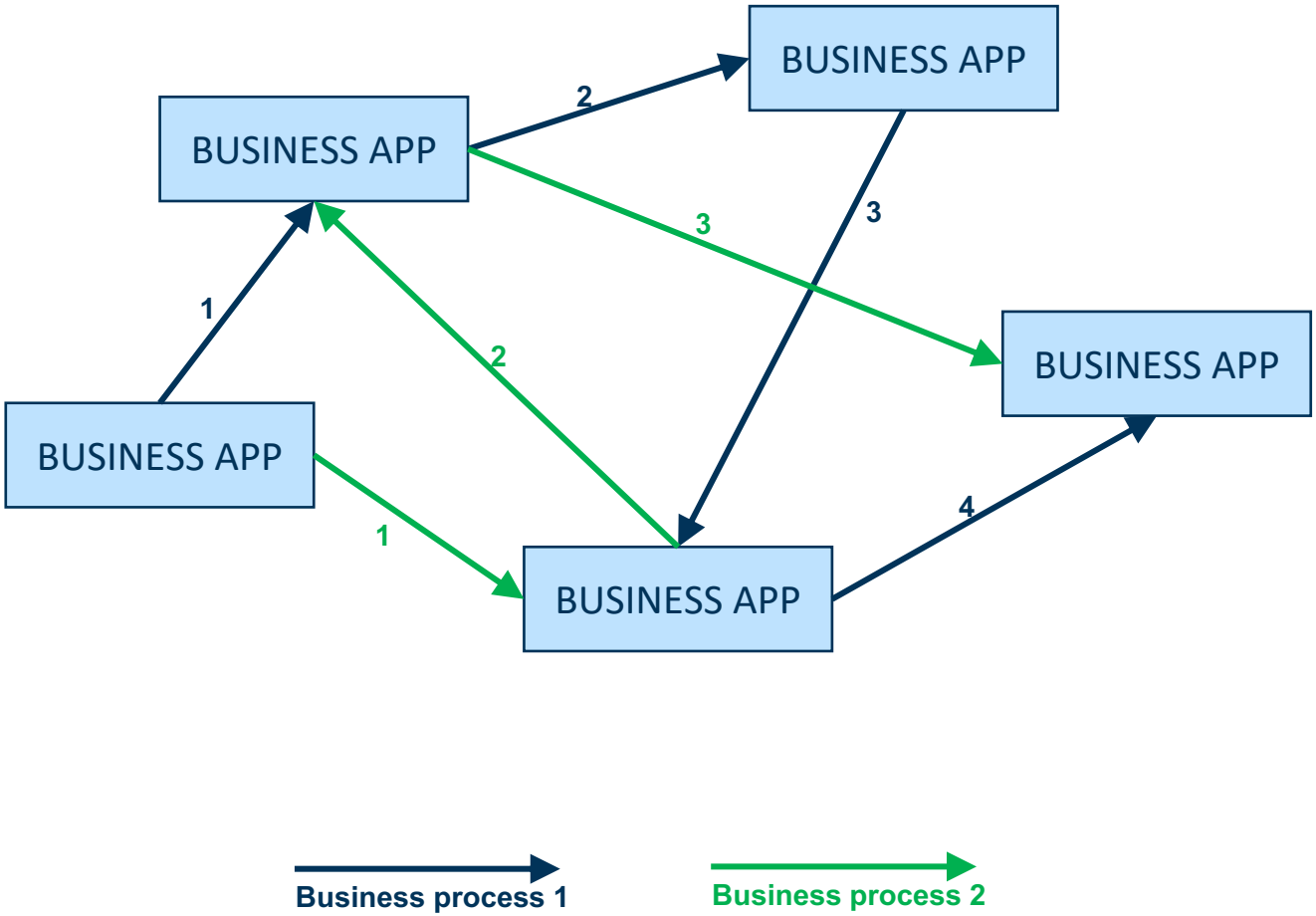


Service-Oriented Architecure



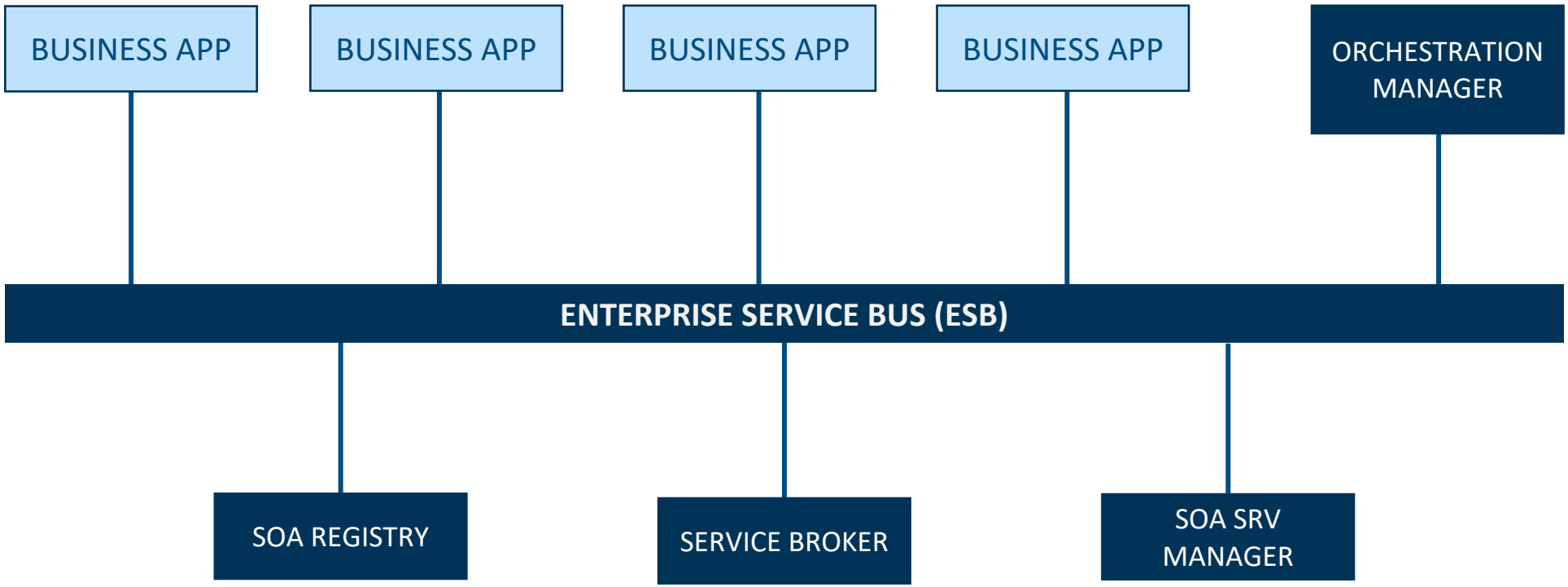
Source: SOA for dummies, free book

Service-Oriented Architecure



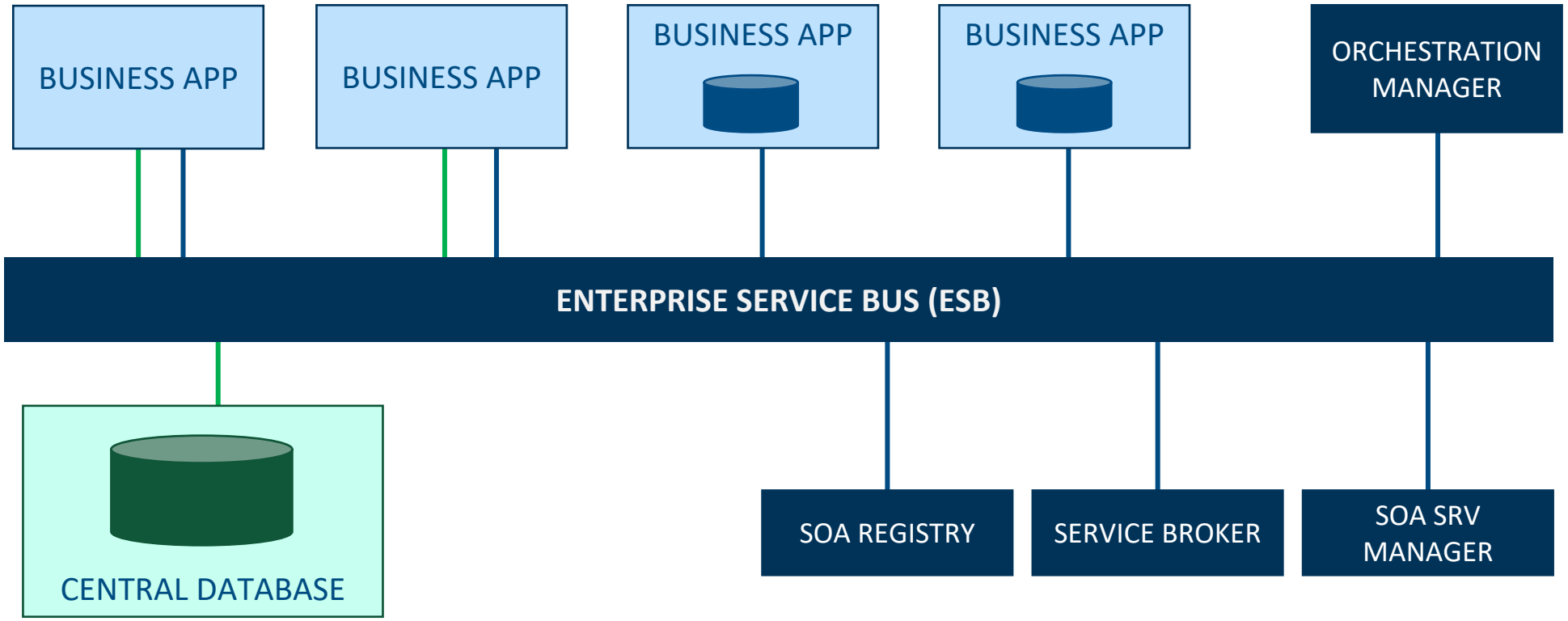
Legacy system

Service-Oriented Architecure

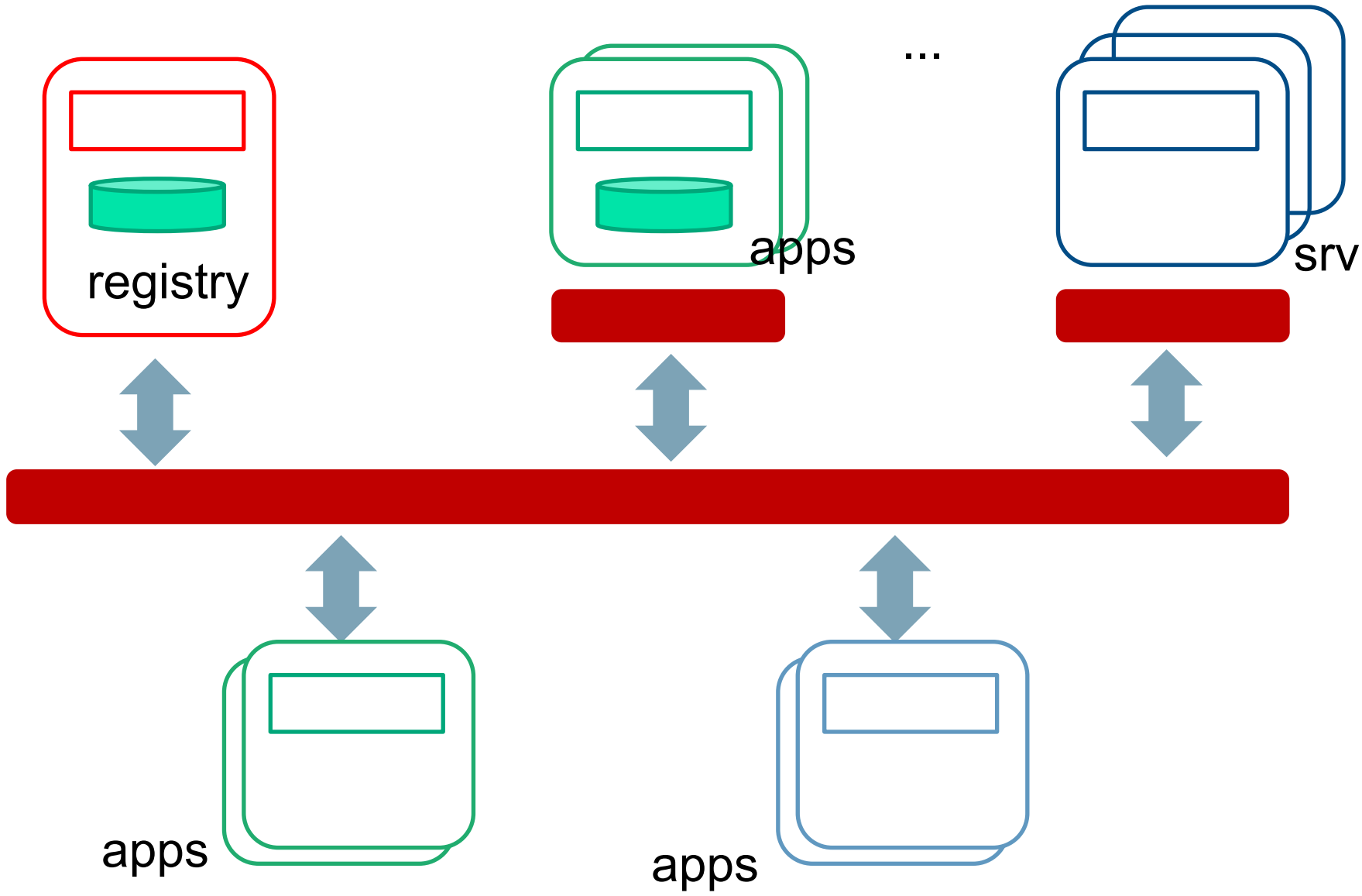


SOA

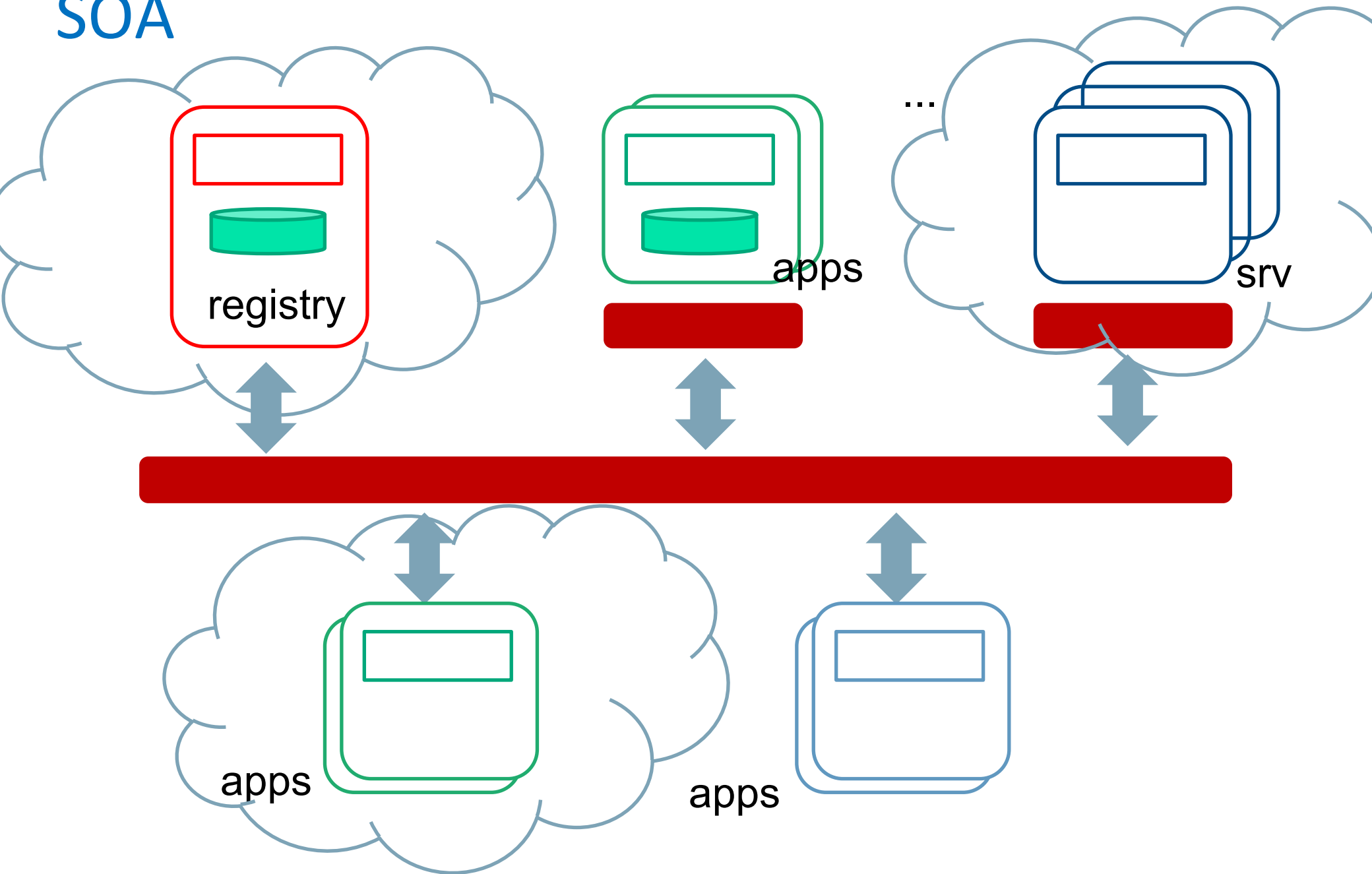
Service-Oriented Architecure



SOA



SOA

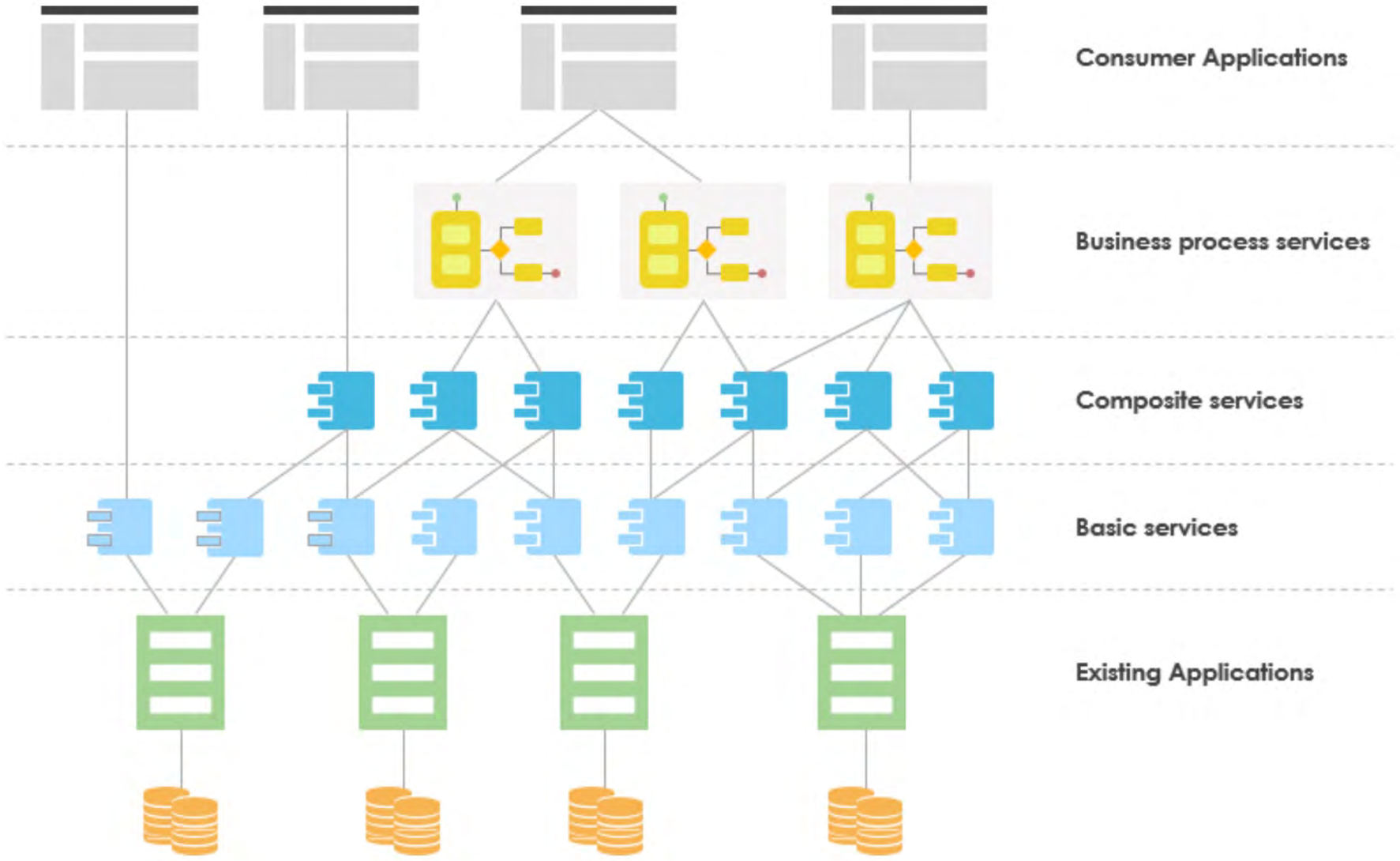


SOA example for a Telecoms company

Existing applications

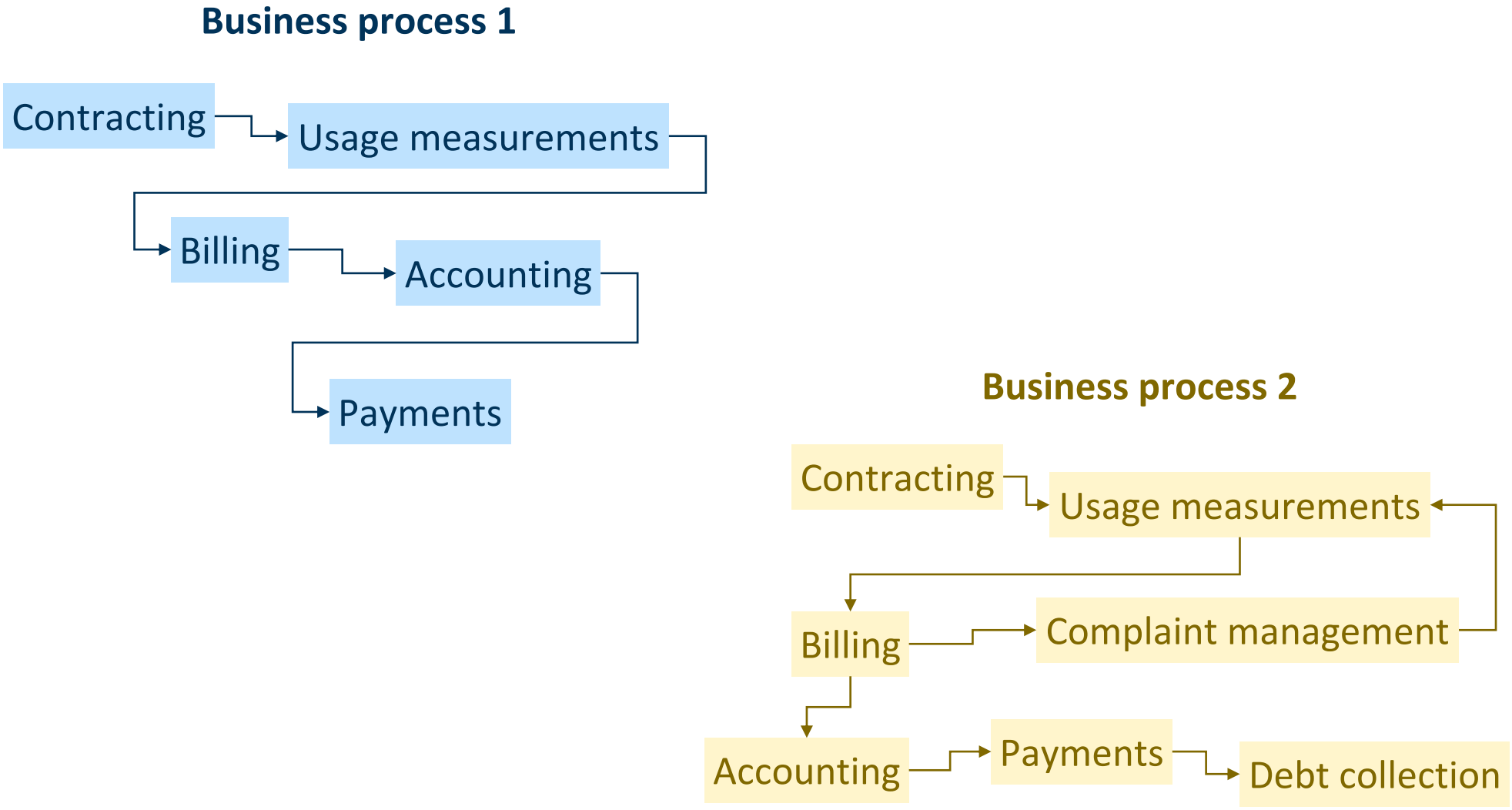
- Complaint management
- Contracting
- Usage measurements
- Billing
- Accounting
- Payments
- Debt collection

SOA: Service-Oriented Architecture

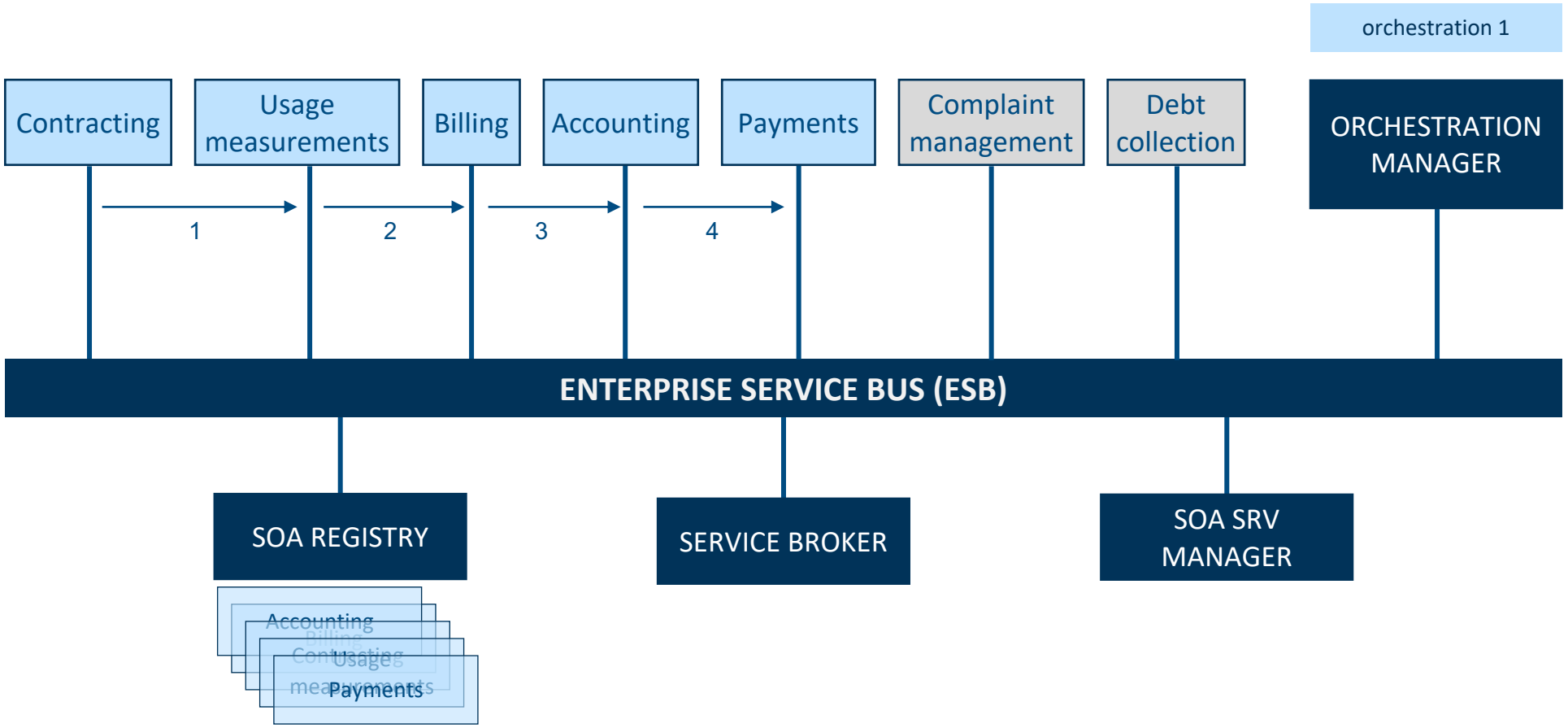


Source: visual-paradigm.com

SOA example for a Telecoms company

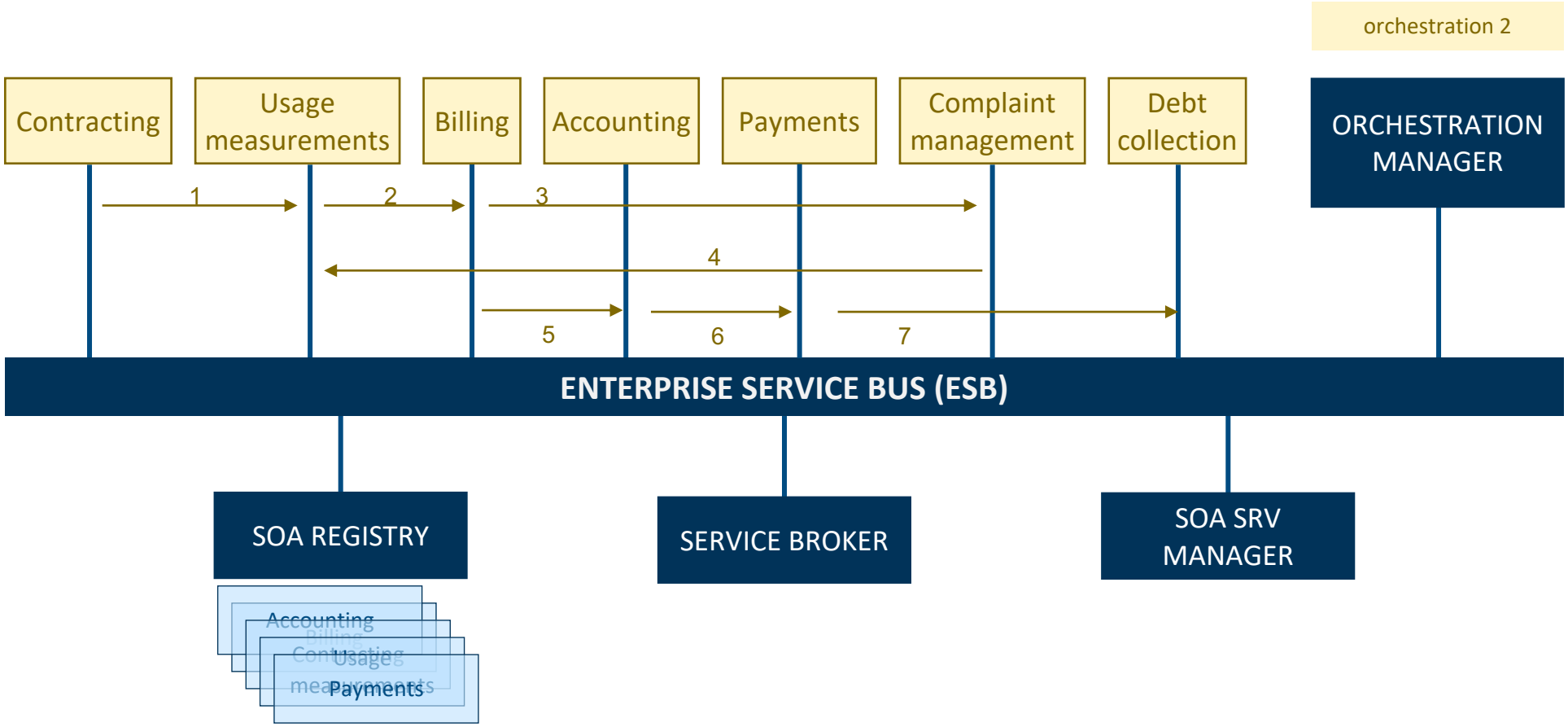


SOA example for a Telecoms company



SOA

SOA example for a Telecoms company



SOA

A SOAML primer

Βασικές έννοιες, παράδειγμα

Credit:
Π.Ζευγολατάκος, Διπλωματική εργασία "*Ενοποίηση συστημάτων blockchain με υπάρχοντα υπολογιστικά οικοσυστήματα με χρήση Service-Oriented Architecture Modeling Language*"

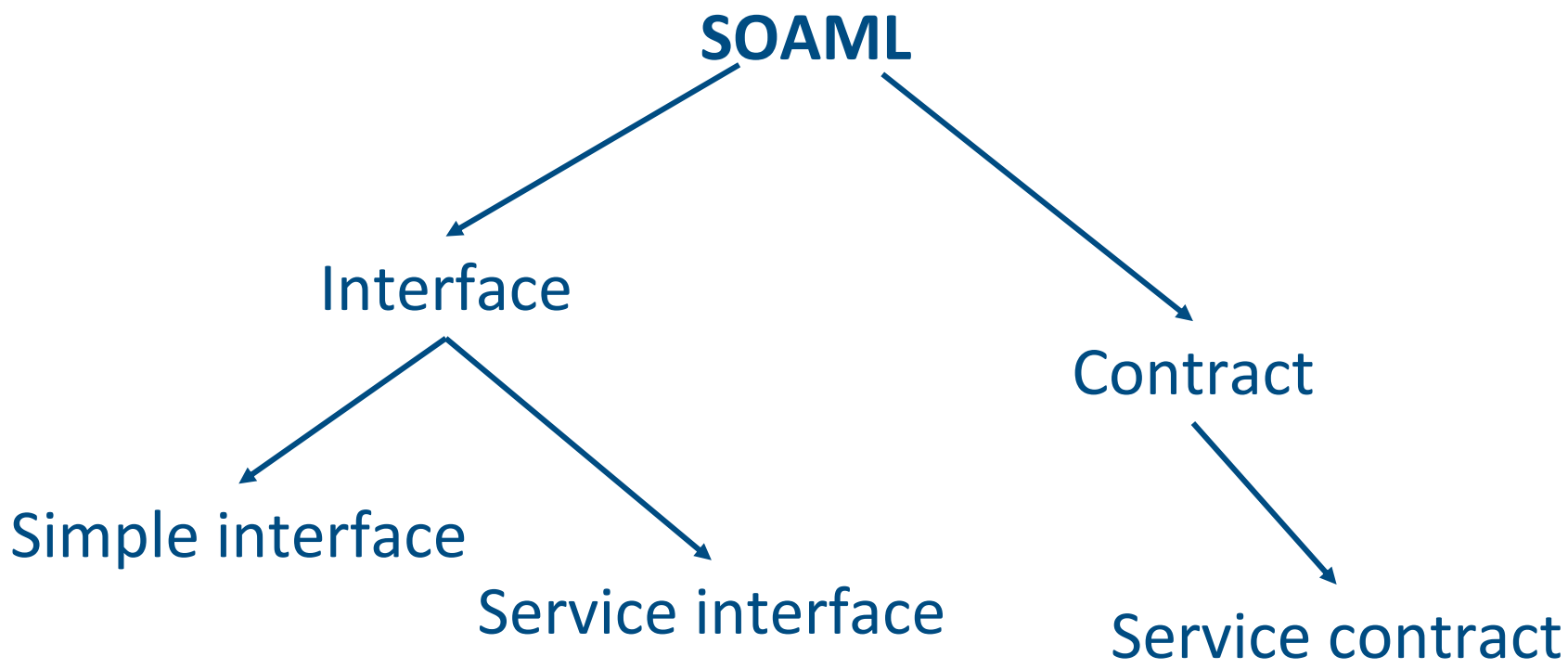
SoaML

Η SOAML προσφέρει έναν τρόπο περιγραφής και τεκμηρίωσης υπηρεσιών μιας αρχιτεκτονικής SOA επεκτείνοντας κατάλληλα την UML

Νέα στερεότυπα:

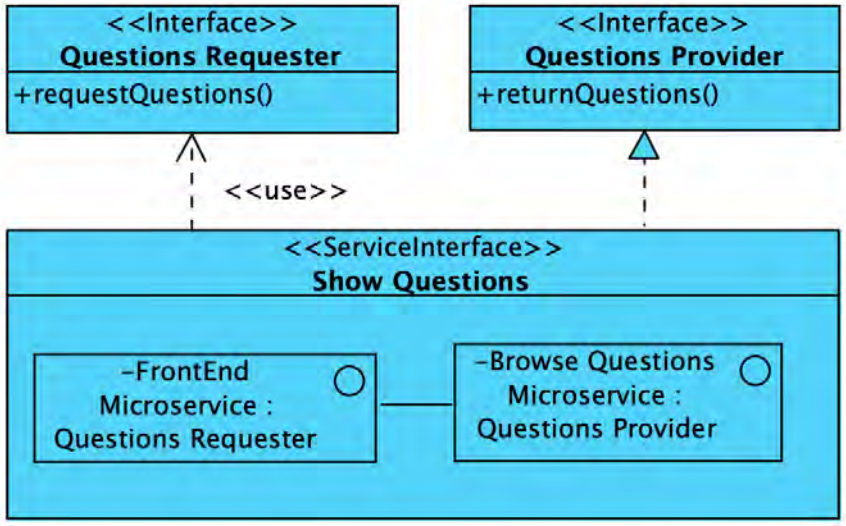
Agent	Attachment	Capability	Consumer
Collaboration	CollaborationUse	Expose	MessageType
Milestone	Participant	Port	Property
Provider	Request	ServiceChannel	ServiceContract
ServiceInterface	Service	ServicesArchitecture	

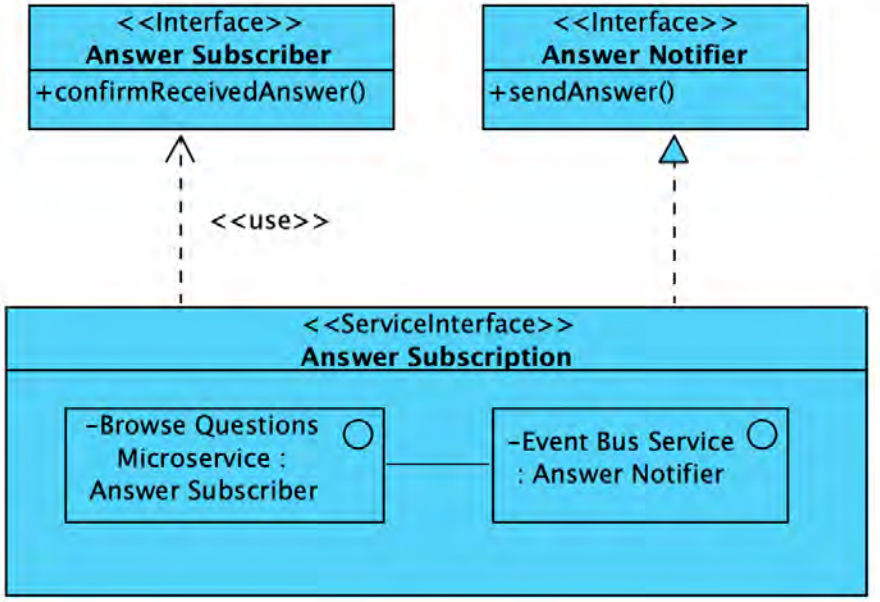
SOAML


SOAML
Interface
Contract
Simple interface
Service interface
Service contract

Διαγράμματα Visual Paradigm

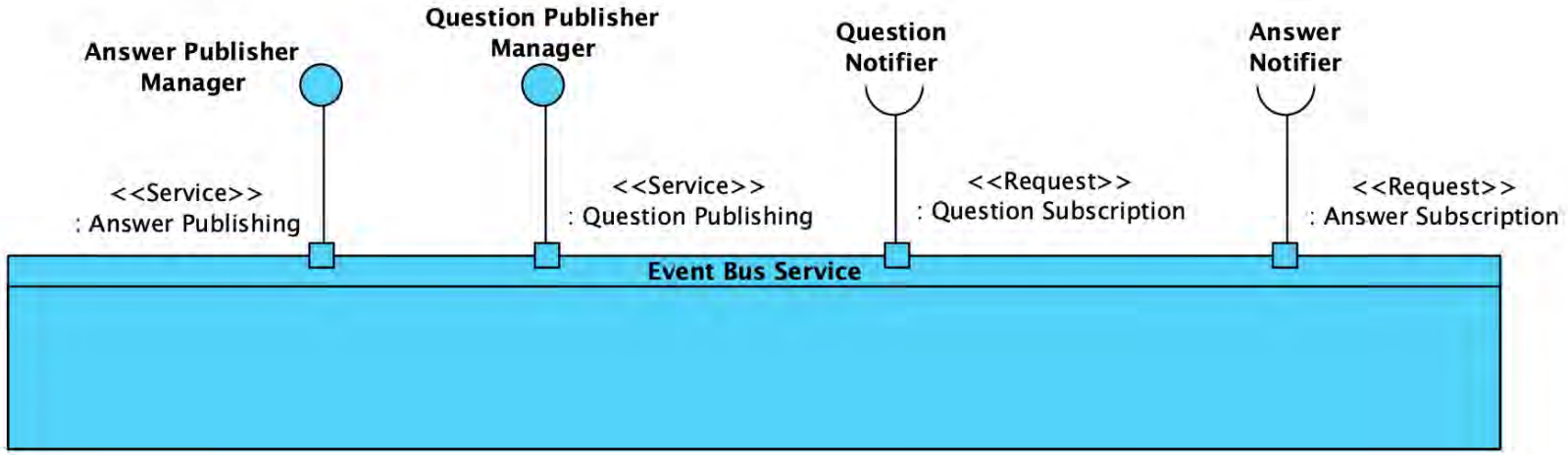
Service Interface Diagram





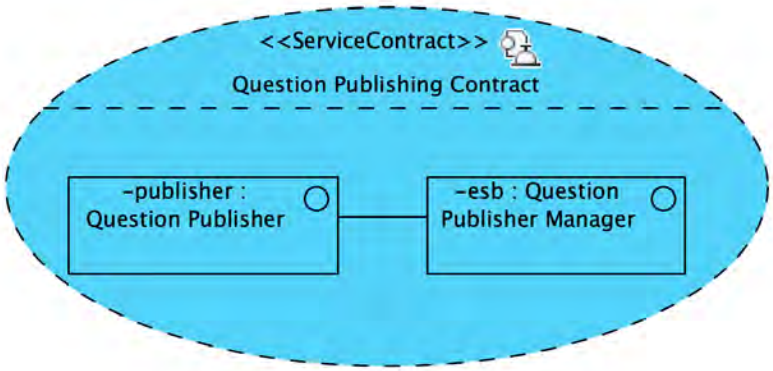
Διαγράμματα Visual Paradigm

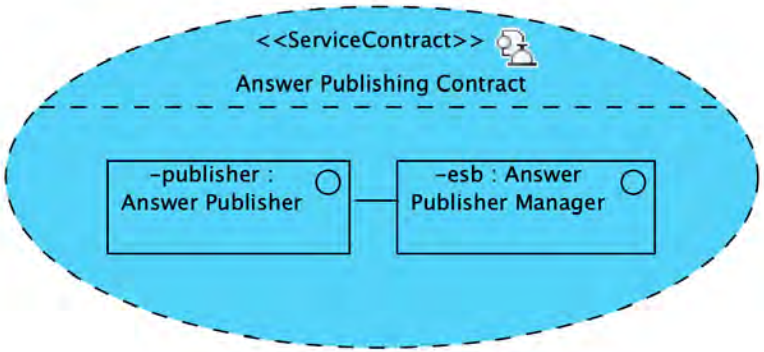
Service Participant Diagram



Διαγράμματα Visual Paradigm

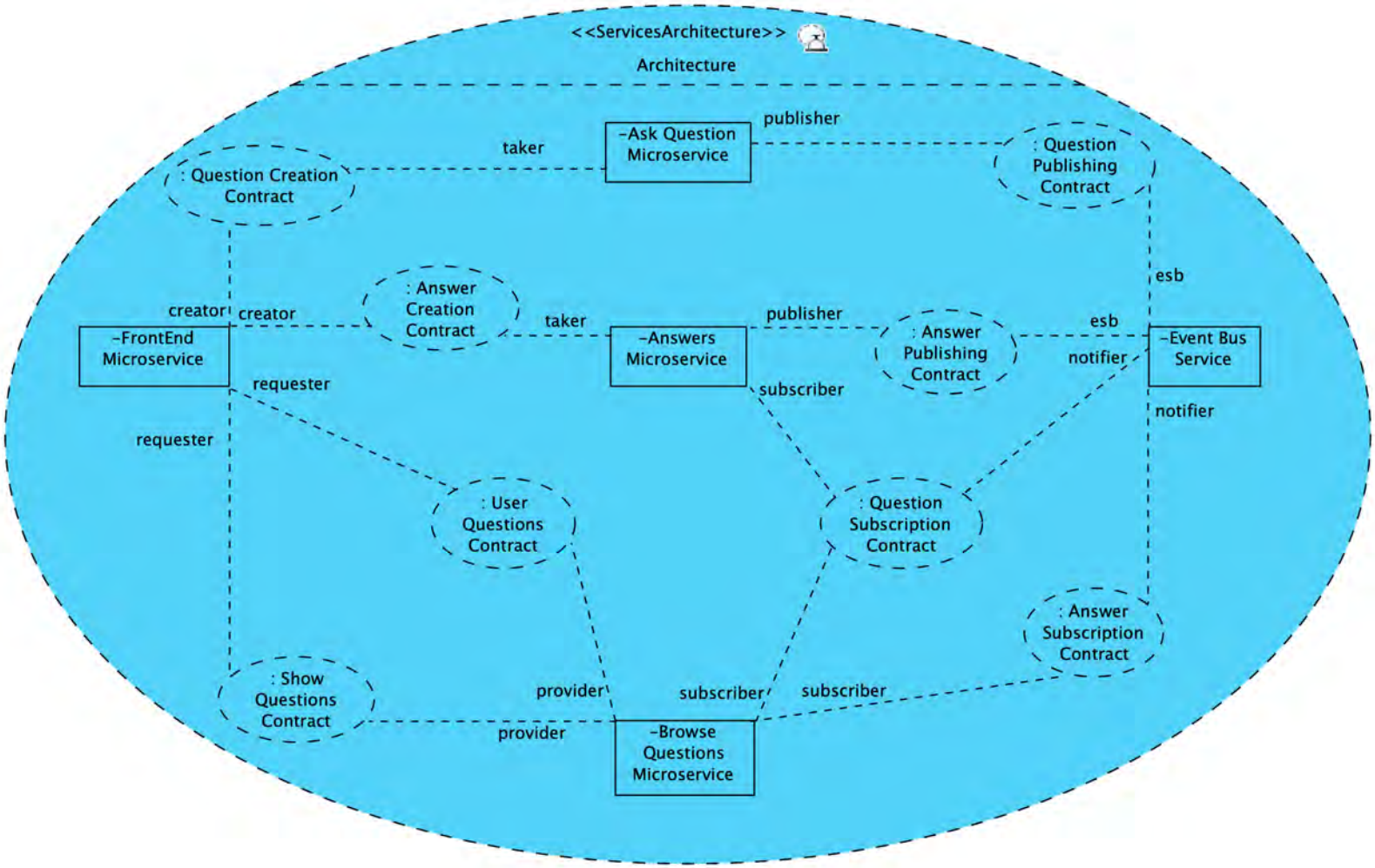
Service Contract Diagram





Διαγράμματα Visual Paradigm

Services Architecture Diagram



Παράδειγμα: Ανάπτυξη εφαρμογής SOA

Παράδειγμα εφαρμογής SOA

Εργασία μαθήματος SaaS 2019-2020

Το “askmeanything” είναι μια εφαρμογή ερωτήσεων - απαντήσεων η οποία θα προσφέρεται ως υπηρεσία λογισμικού (SaaS) σε υποθετικούς πελάτες όπως εταιρίες που προσφέρουν υπηρεσίες τύπου help desk, εταιρίες που περιγράφουν τις εσωτερικές τους διαδικασίες με τη μορφή ερωτήσεων-απαντήσεων, εκπαίδευση κ.ά. Στο πλαίσιο της εργασίας θα αναπτυχθεί μια απλουστευμένη εκδοχή της εφαρμογής η οποία εκτελεί τις ακόλουθες λειτουργίες:

- Δημιουργία ερώτησης
- Χαρακτηρισμός με λέξεις-κλειδιά
- Απάντηση ερώτησης
- Περιήγηση σε ερωταπαντήσεις
- Στατιστικά ερωτήσεων με λέξεις-κλειδιά
- Στατιστικά ερωτήσεων με ημερομηνίες
- Sign up, sign in

Use Case

Λειτουργίες

- Δημιουργία ερώτησης
- Χαρακτηρισμός με λέξεις-κλειδιά
- Απάντηση ερώτησης
- Περιήγηση σε ερωταπαντήσεις

Use Case

Υπηρεσίες

- Ask Questions
- Answers
- Browse Questions
- Frontend
- Enterprise Service Bus

Υλοποίηση

Enterprise Service Bus

- Kafka (83.212.78.171)
- Topics
  - QUESTION
  - ANSWER
- log.retention.hours = -1
- Διαφορετικά consumer groups

Υλοποίηση

Ask Questions Service

- Αποστολή ερωτήσεων στο topic QUESTION
- “/”: POST Request
- QuestionMessage:

```
{ qname: string,
  qtext: string,
qkeywords: string
}
```

Υλοποίηση

Answers Service

- Αποστολή απαντήσεων στο topic ANSWER
- Προβολή ερώτησης → “answers-group”
- “/answer/:id”: POST Request
- “/question/:id”: GET Request
- AnswerMessage:

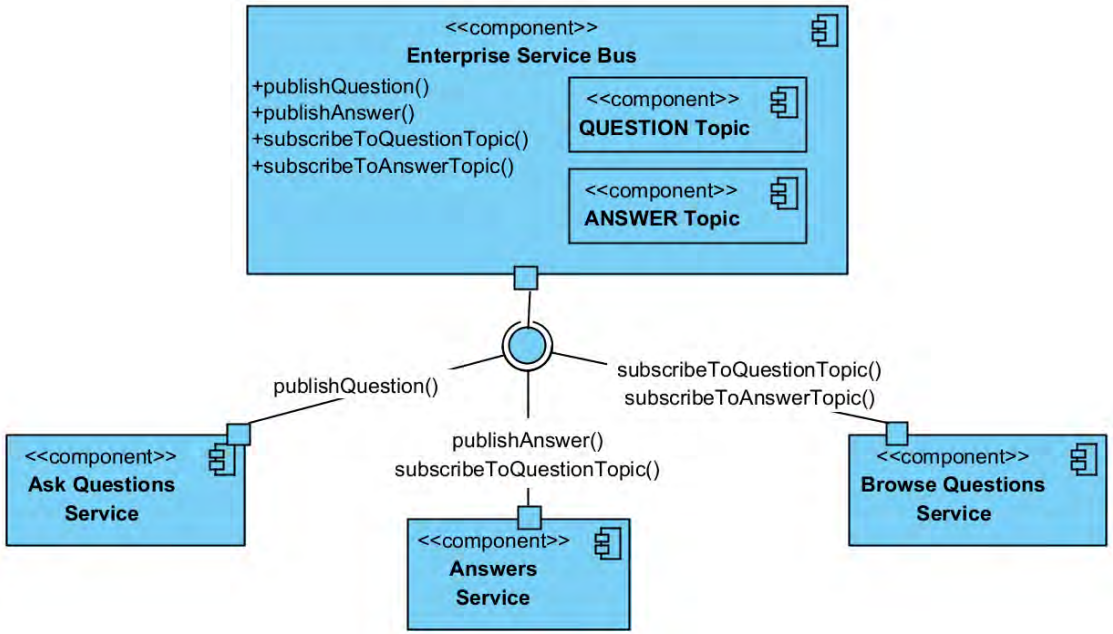
```
{ answer: string,
 questionId: int,
    answerId: int
             }
```

Υλοποίηση

Browse Questions Service

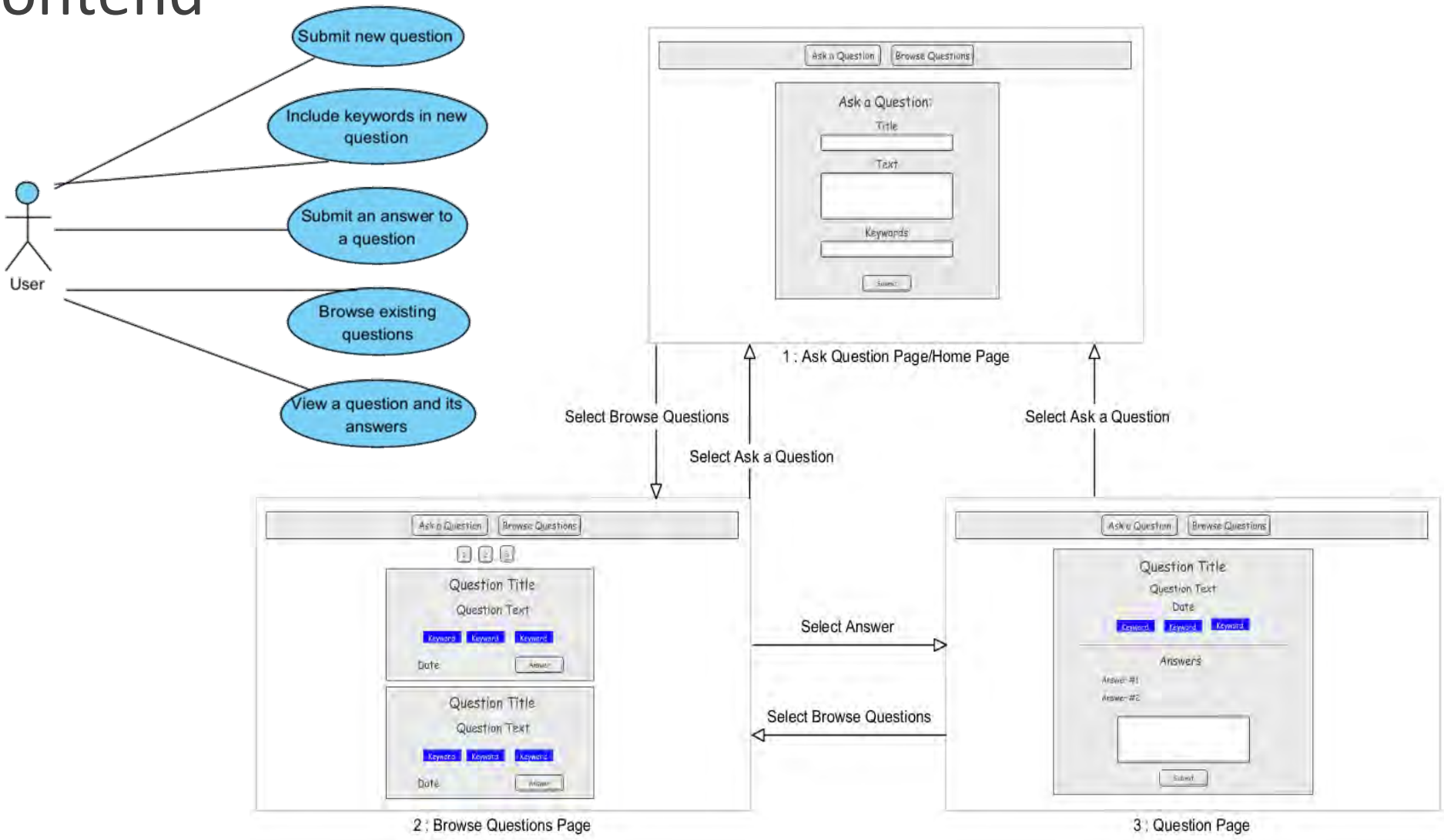
- Προβολή ερωτήσεων → “browse-questions-group”
- “/show”: POST Request

Υλοποίηση

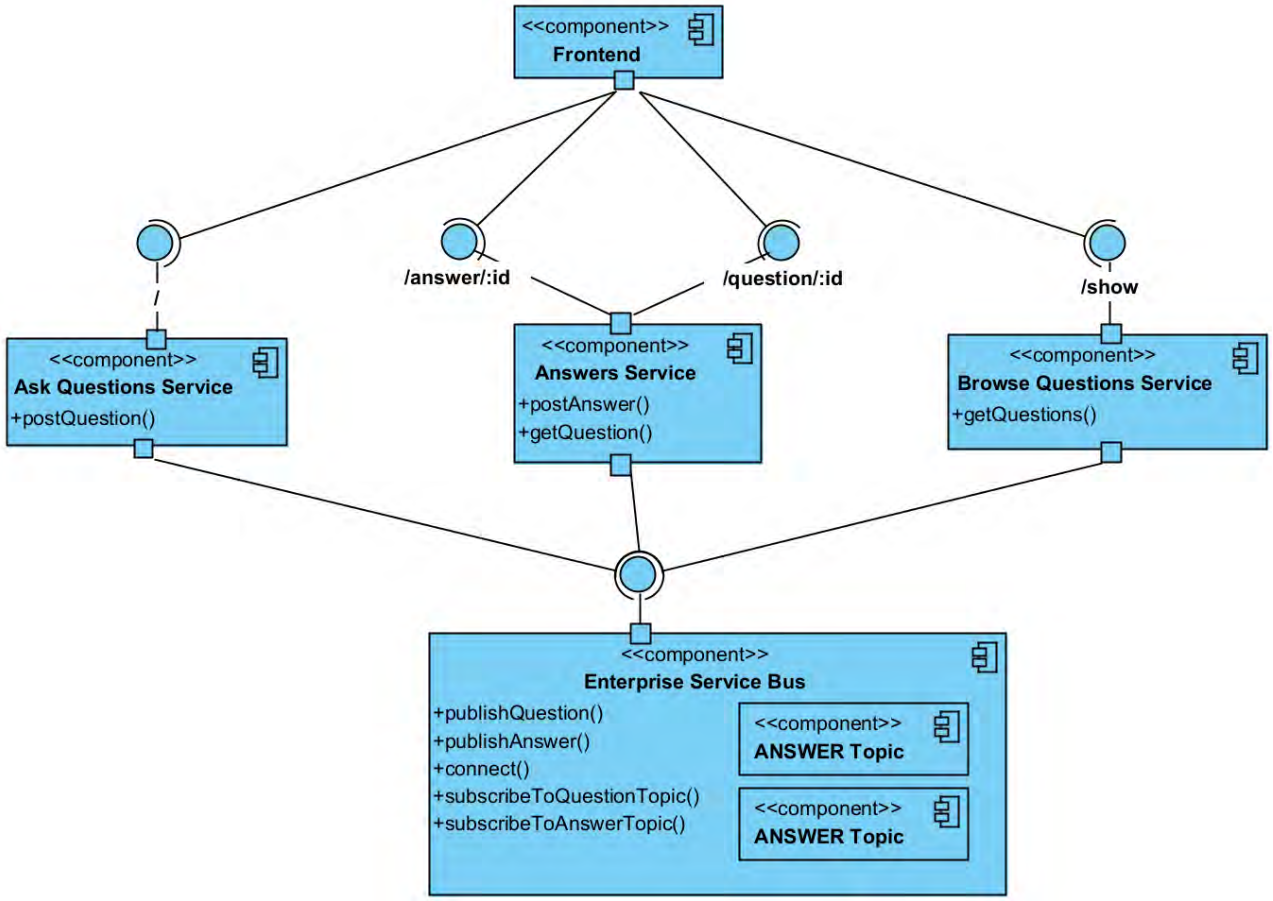

<<component>>
Enterprise Service Bus
+publishQuestion()
+publishAnswer()
+subscribeToQuestionTopic()
+subscribeToAnswerTopic()
<<component>>
QUESTION Topic
<<component>>
ANSWER Topic
publishQuestion()
subscribeToQuestionTopic()
subscribeToAnswerTopic()
publishAnswer()
subscribeToQuestionTopic()
<<component>>
Ask Questions Service
<<component>>
Answers Service
<<component>>
Browse Questions Service

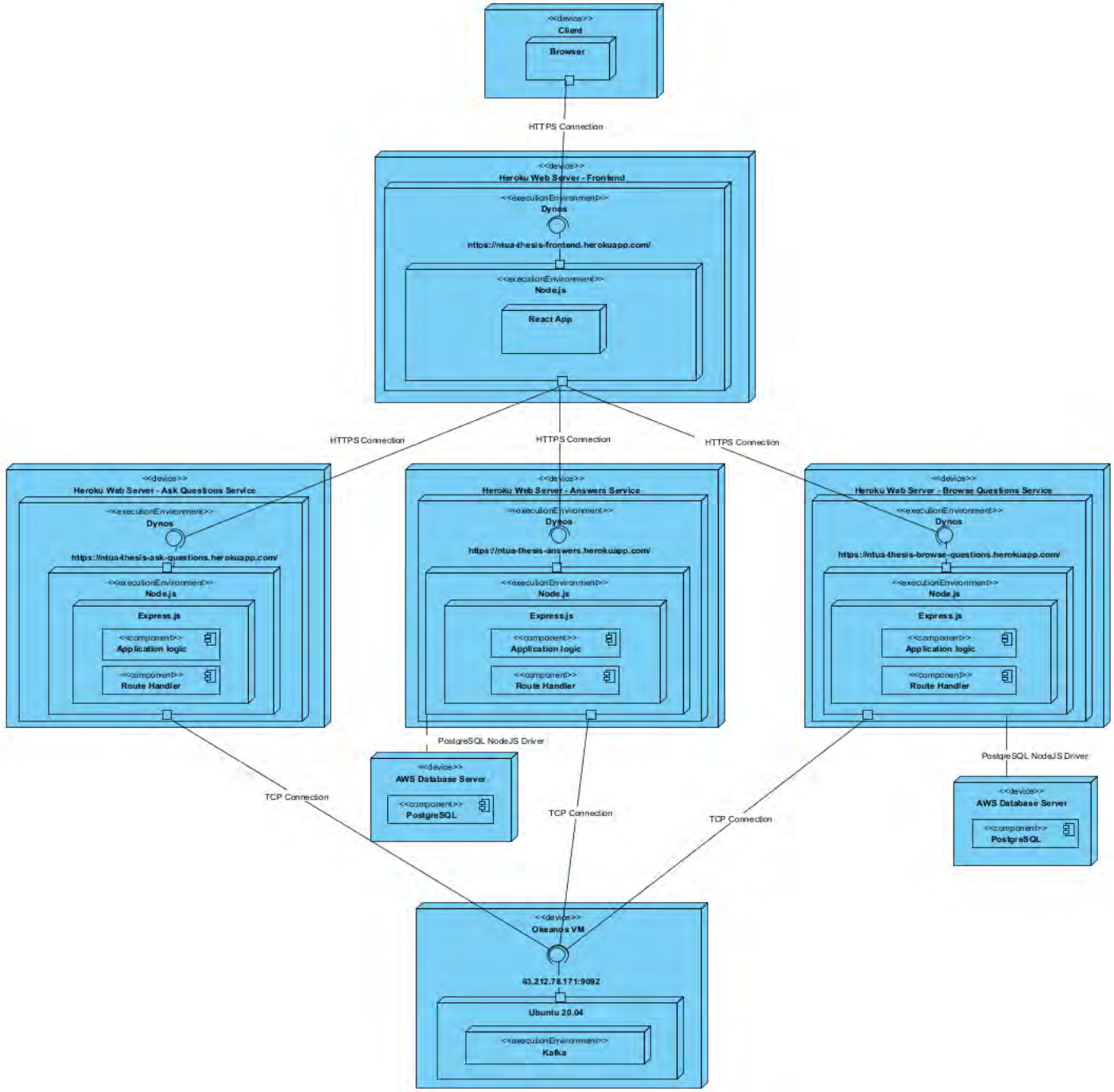
Υλοποίηση

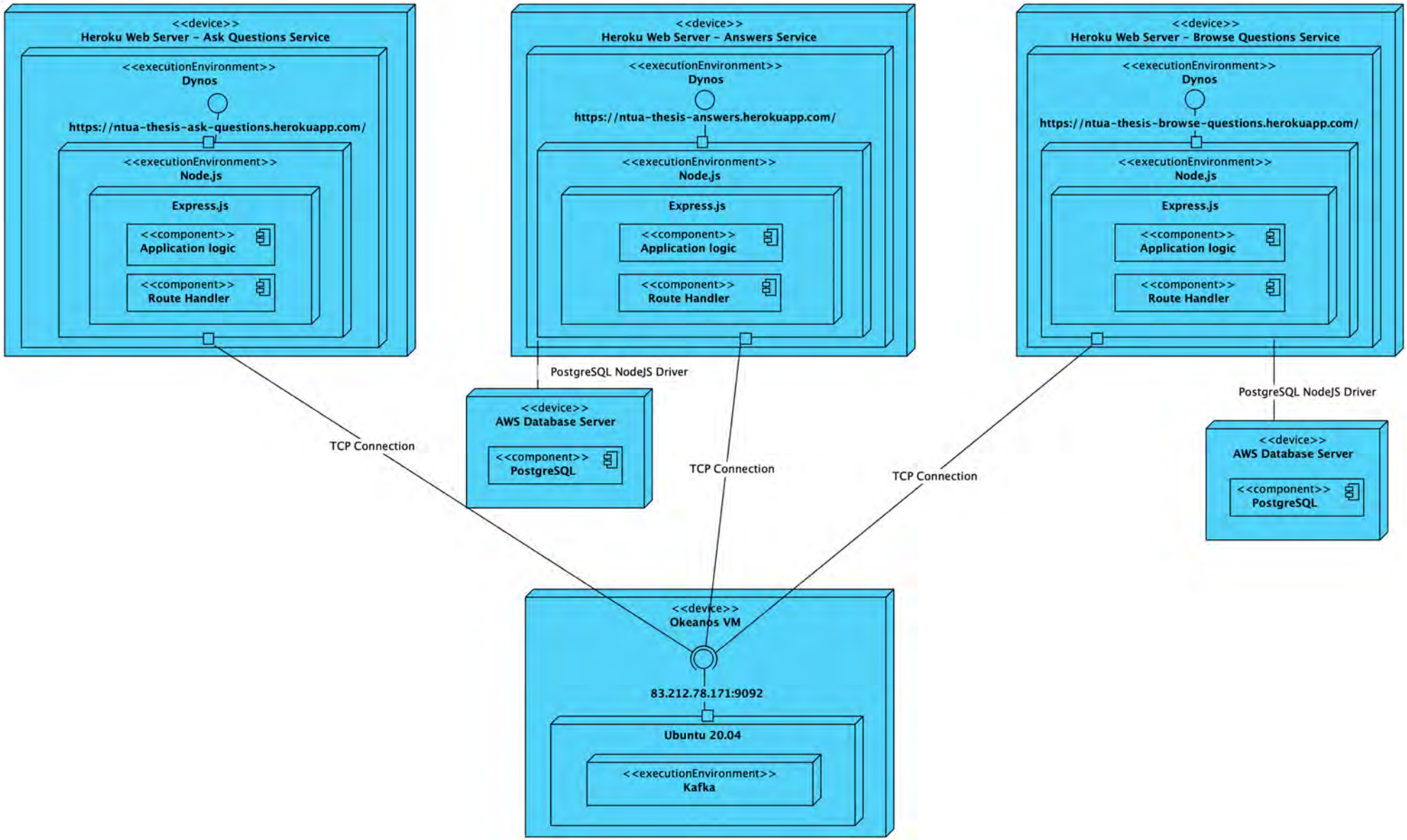
Frontend

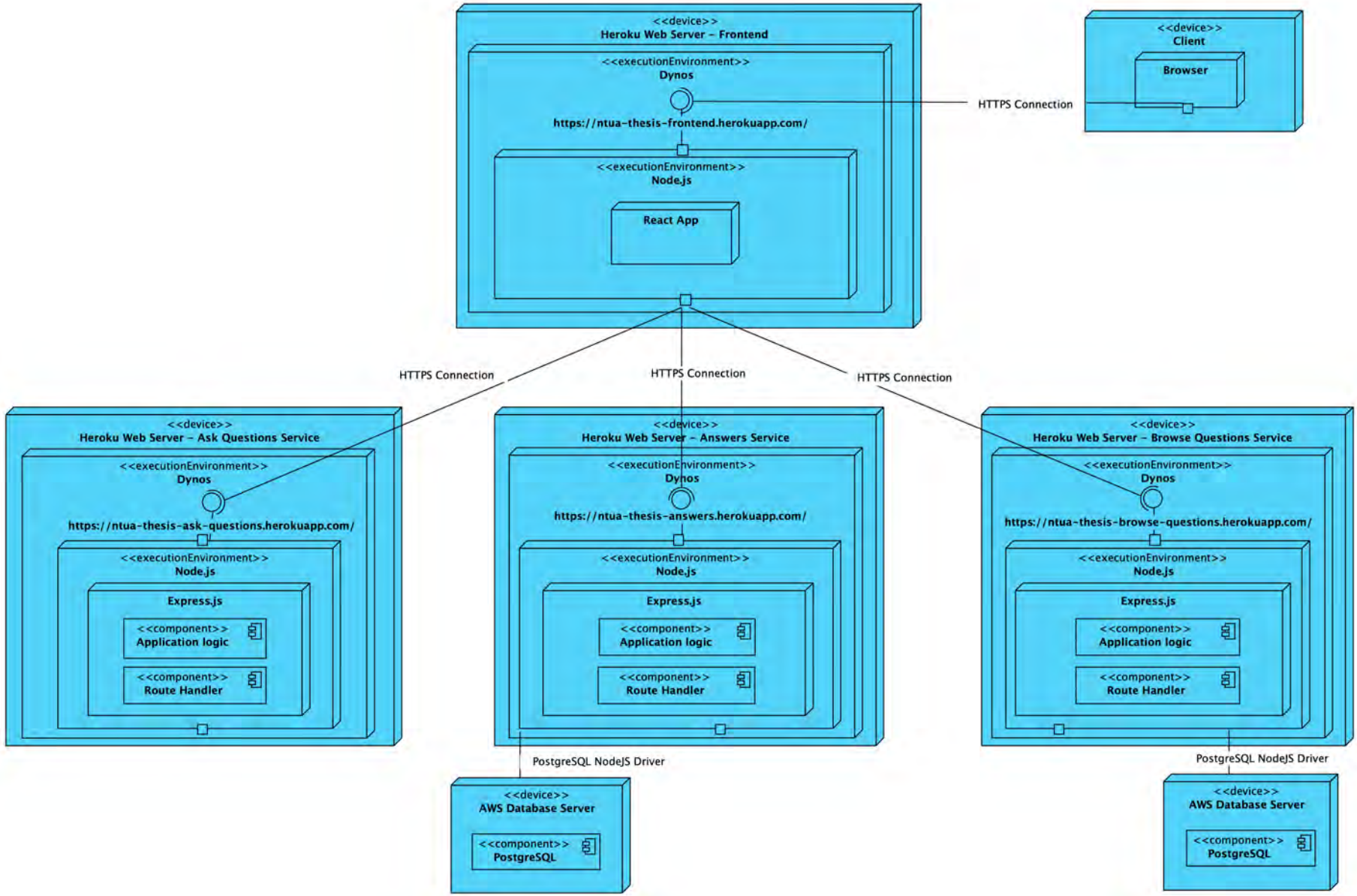


Υλοποίηση


<<component>>
Frontend
/
/answer/:id
/question/:id
/show
<<component>>
Ask Questions Service
+postQuestion()
<<component>>
Answers Service
+postAnswer()
+getQuestion()
<<component>>
Browse Questions Service
+getQuestions()
<<component>>
Enterprise Service Bus
+publishQuestion()
+publishAnswer()
+connect()
+subscribeToQuestionTopic()
+subscribeToAnswerTopic()
<<component>>
ANSWER Topic
<<component>>
ANSWER Topic


<<device>>
Client
Browser
HTTPS Connection
<<device>>
Heroku Web Server - Frontend
<<executionEnvironment>>
Dynos
https://ntua-thesis-frontend.herokuapp.com/
<<executionEnvironment>>
Node.js
React App
HTTPS Connection
HTTPS Connection
HTTPS Connection
<<device>>
Heroku Web Server - Ask Questions Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-ask-questions.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
<<device>>
Heroku Web Server - Answers Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-answers.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
<<device>>
Heroku Web Server - Browse Questions Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-browse-questions.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
PostgreSQL NodeJS Driver
PostgreSQL NodeJS Driver
<<device>>
AWS Database Server
<<component>>
PostgreSQL
<<device>>
AWS Database Server
<<component>>
PostgreSQL
TCP Connection
TCP Connection
TCP Connection
<<device>>
Okeanos VM
83.212.78.171:9092
Ubuntu 20.04
<<executionEnvironment>>
Kafka

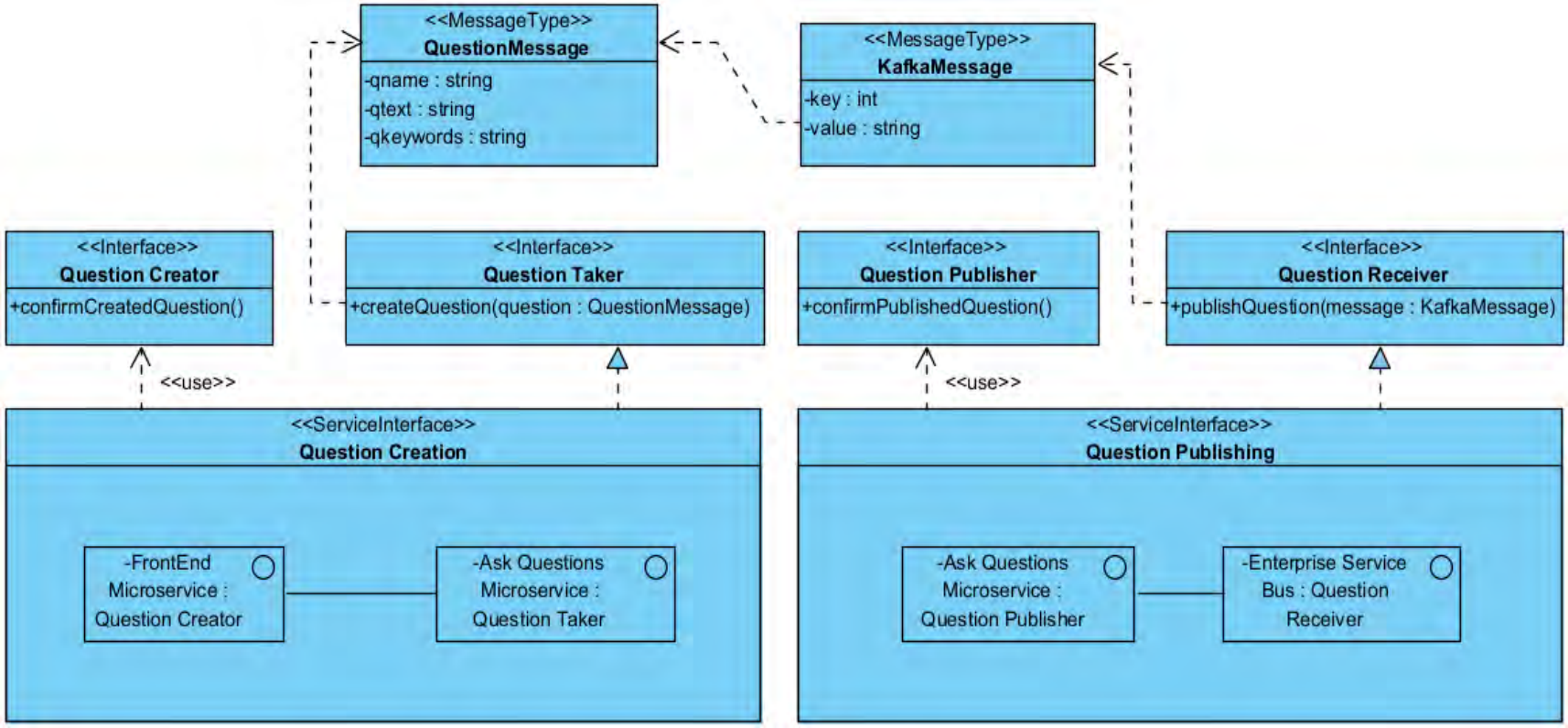

<<device>>
Heroku Web Server - Ask Questions Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-ask-questions.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
<<device>>
Heroku Web Server - Answers Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-answers.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
<<device>>
Heroku Web Server - Browse Questions Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-browse-questions.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
PostgreSQL NodeJS Driver
PostgreSQL NodeJS Driver
<<device>>
AWS Database Server
<<component>>
PostgreSQL
<<device>>
AWS Database Server
<<component>>
PostgreSQL
TCP Connection
TCP Connection
TCP Connection
<<device>>
Okeanos VM
83.212.78.171:9092
Ubuntu 20.04
<<executionEnvironment>>
Kafka


<<device>>
Heroku Web Server – Frontend
<<executionEnvironment>>
Dynos
https://ntua-thesis-frontend.herokuapp.com/
<<executionEnvironment>>
Node.js
React App
<<device>>
Client
Browser
HTTPS Connection
HTTPS Connection
HTTPS Connection
HTTPS Connection
<<device>>
Heroku Web Server – Ask Questions Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-ask-questions.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
<<device>>
Heroku Web Server – Answers Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-answers.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
<<device>>
Heroku Web Server – Browse Questions Service
<<executionEnvironment>>
Dynos
https://ntua-thesis-browse-questions.herokuapp.com/
<<executionEnvironment>>
Node.js
Express.js
<<component>>
Application logic
<<component>>
Route Handler
PostgreSQL NodeJS Driver
PostgreSQL NodeJS Driver
<<device>>
AWS Database Server
<<component>>
PostgreSQL
<<device>>
AWS Database Server
<<component>>
PostgreSQL

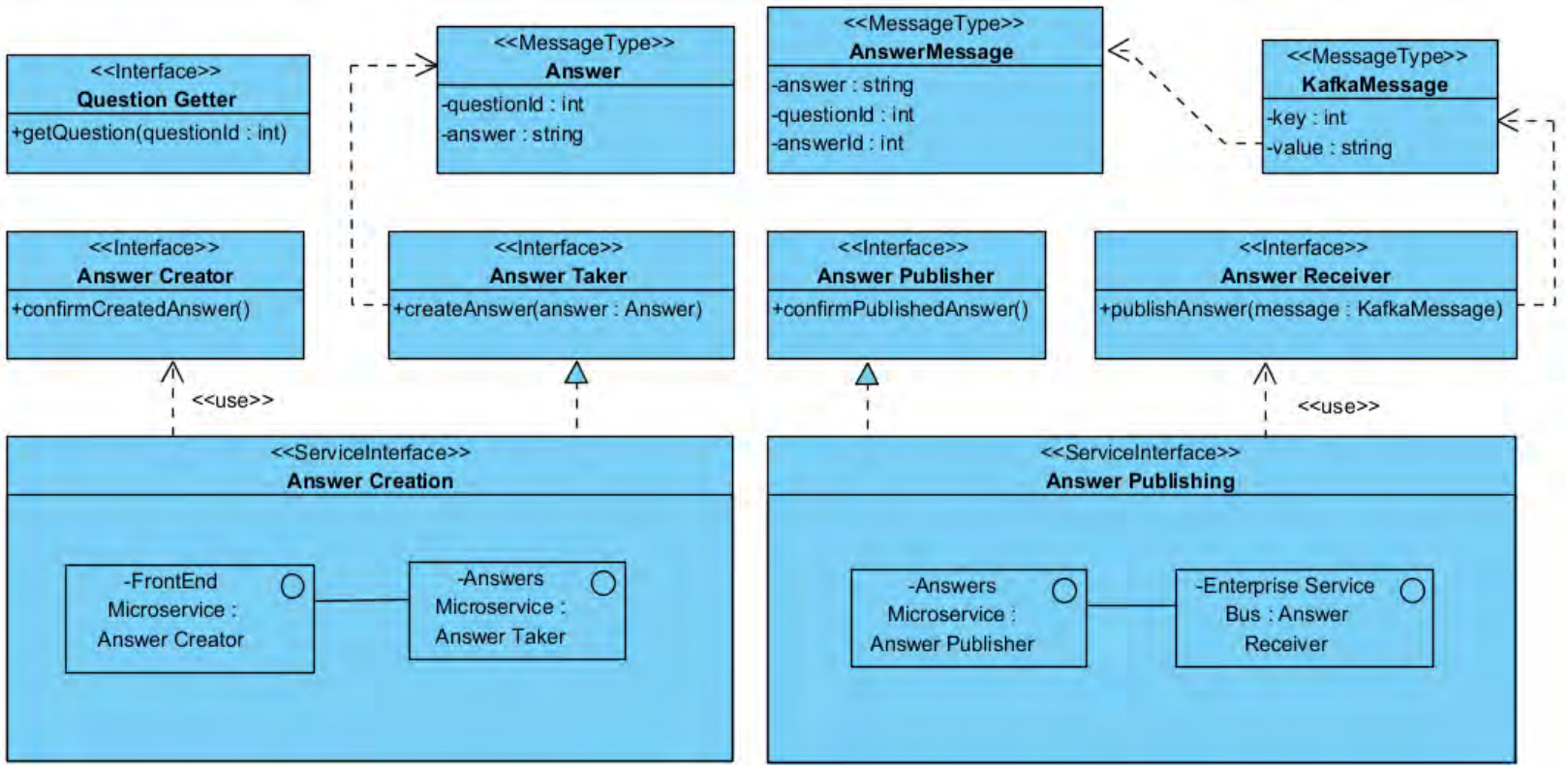
Παράδειγμα: Τεκμηρίωση με SoaML

Service Interface Diagrams

Ask Questions Service Interface Diagram

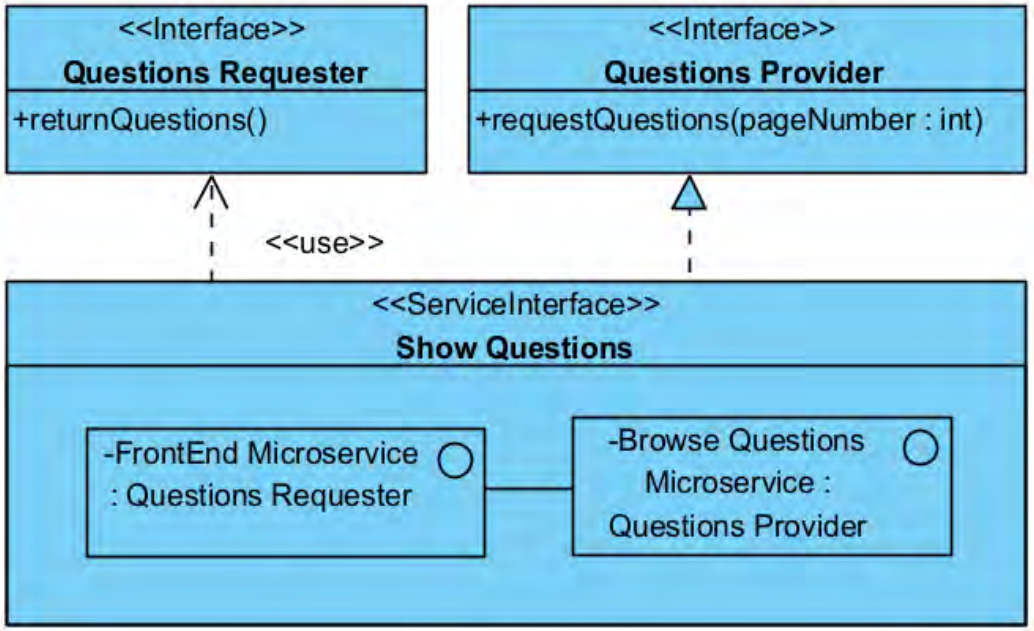


Service Interface Diagrams



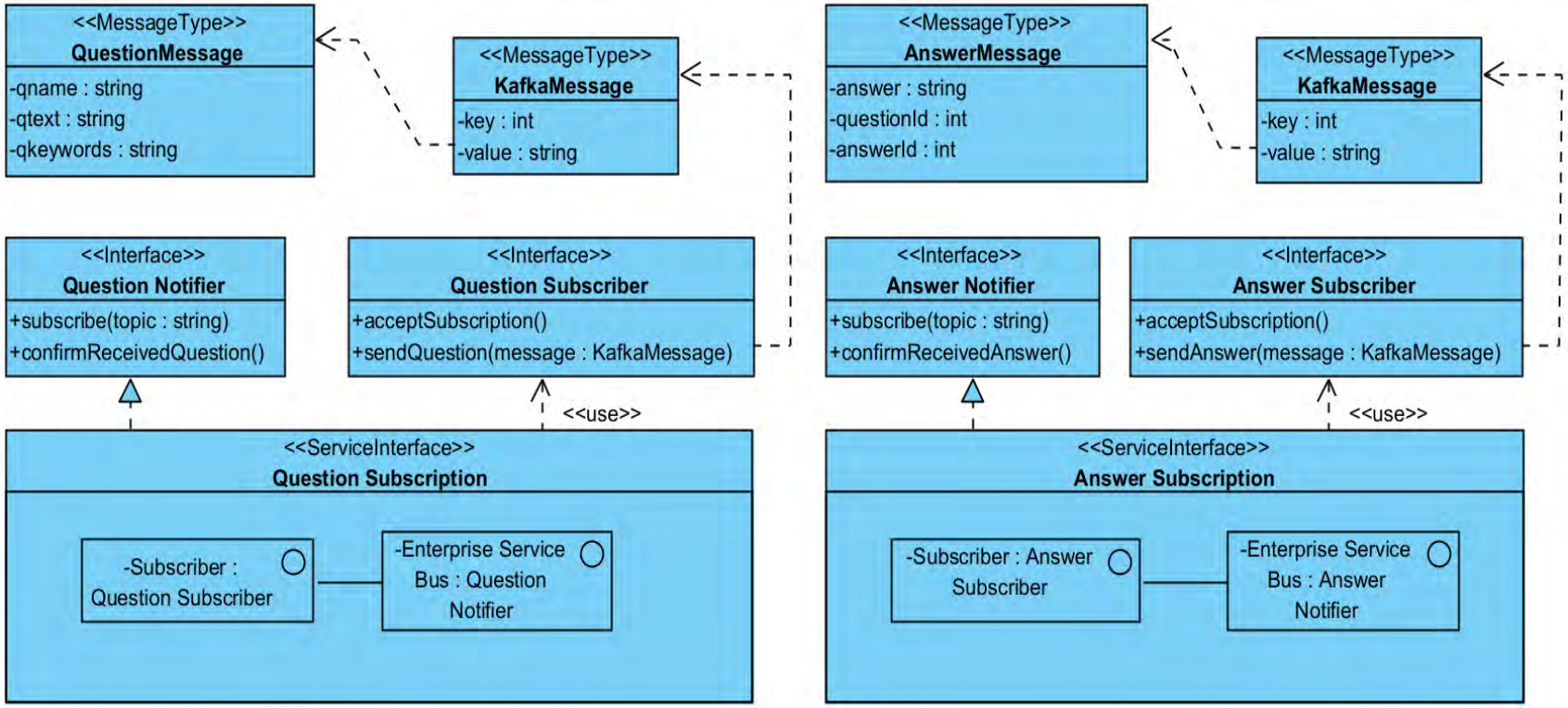
Answers Service Interface Diagram

Service Interface Diagrams



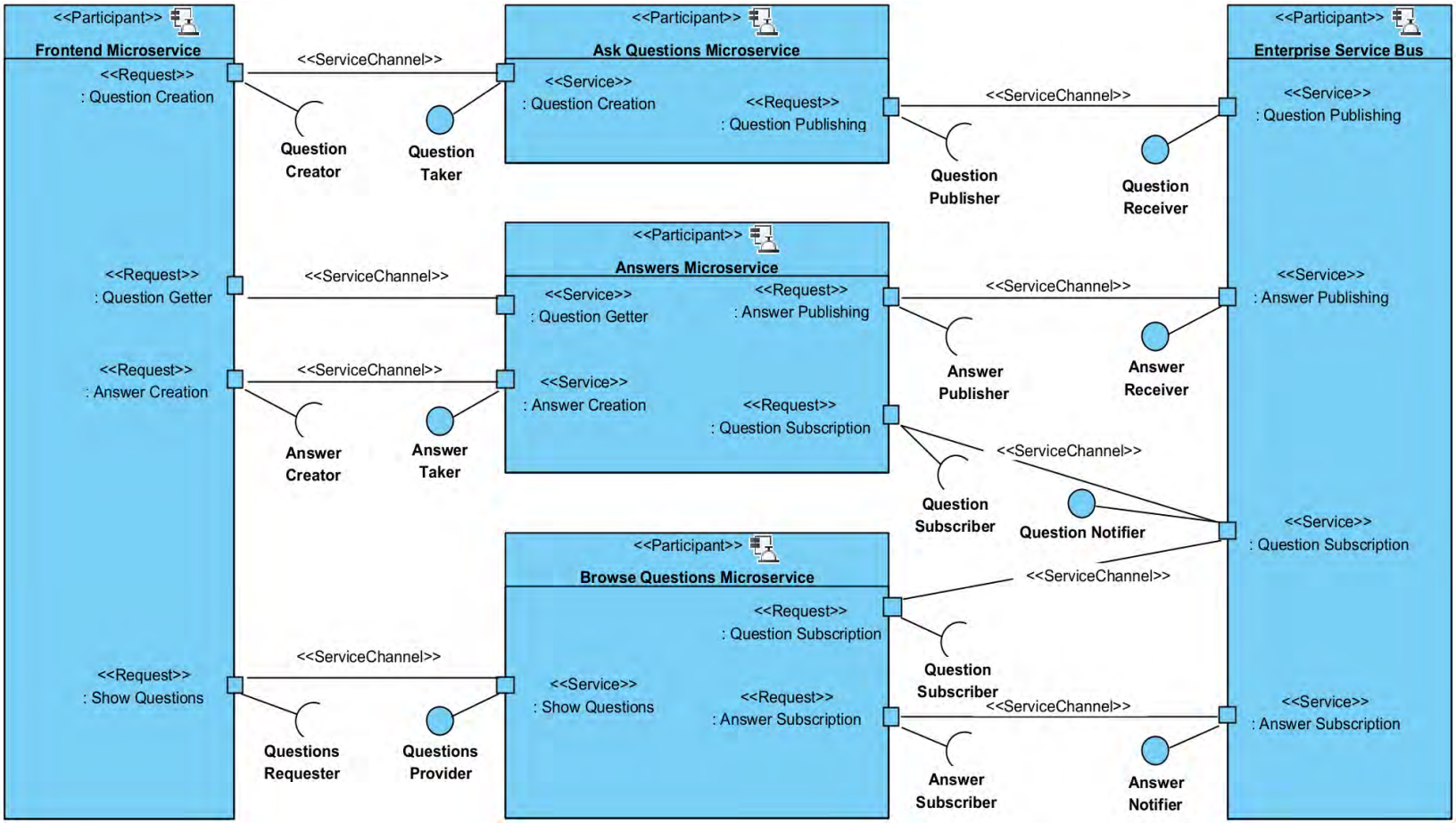
Browse Questions Service Interface Diagram

Service Interface Diagrams



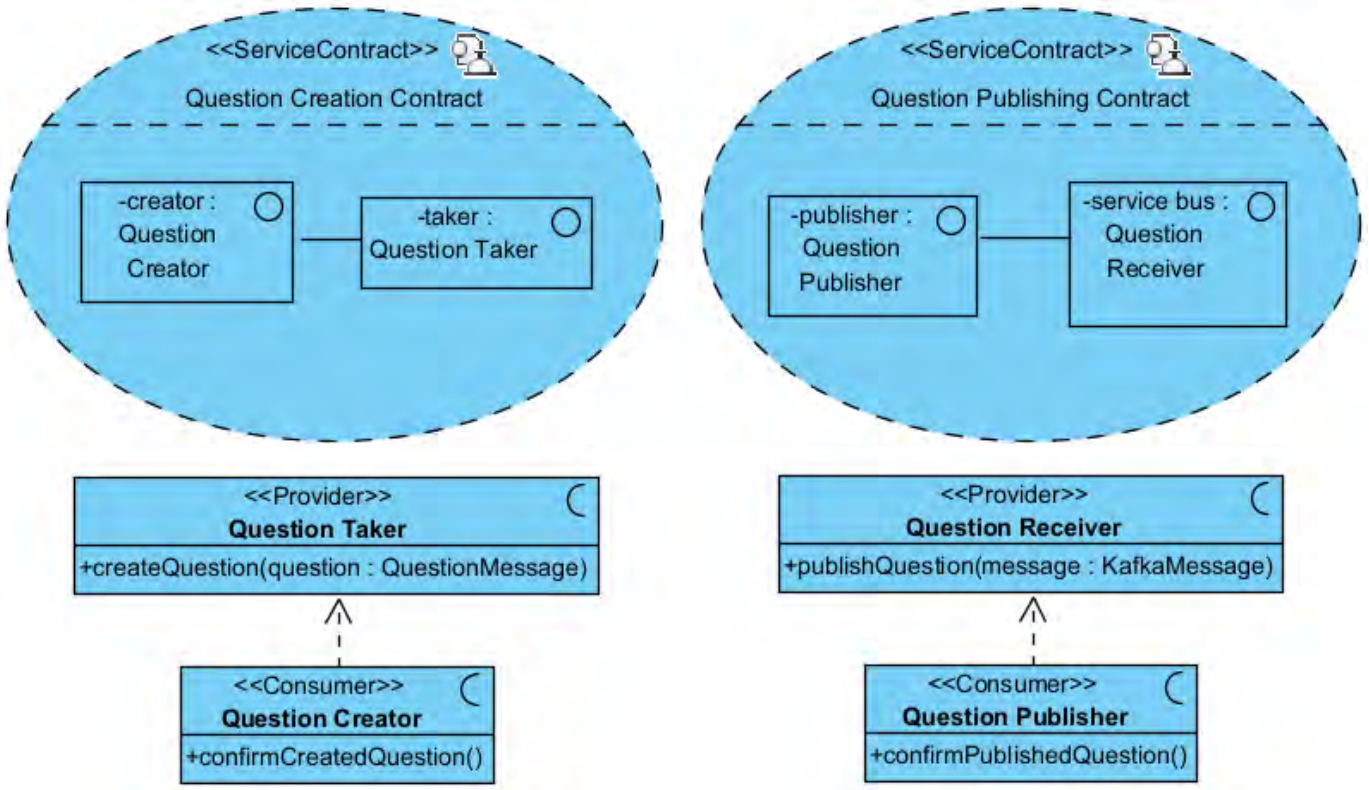
Enterprise Service Bus Service Interface Diagram

Service Participant Diagram



Microservices Service Participant Diagram

Service Contract Diagrams



Ask Questions Service Contract Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Question Creation service.
The taker : Question Taker is the Provider and the creator : Question Creator is the Consumer.

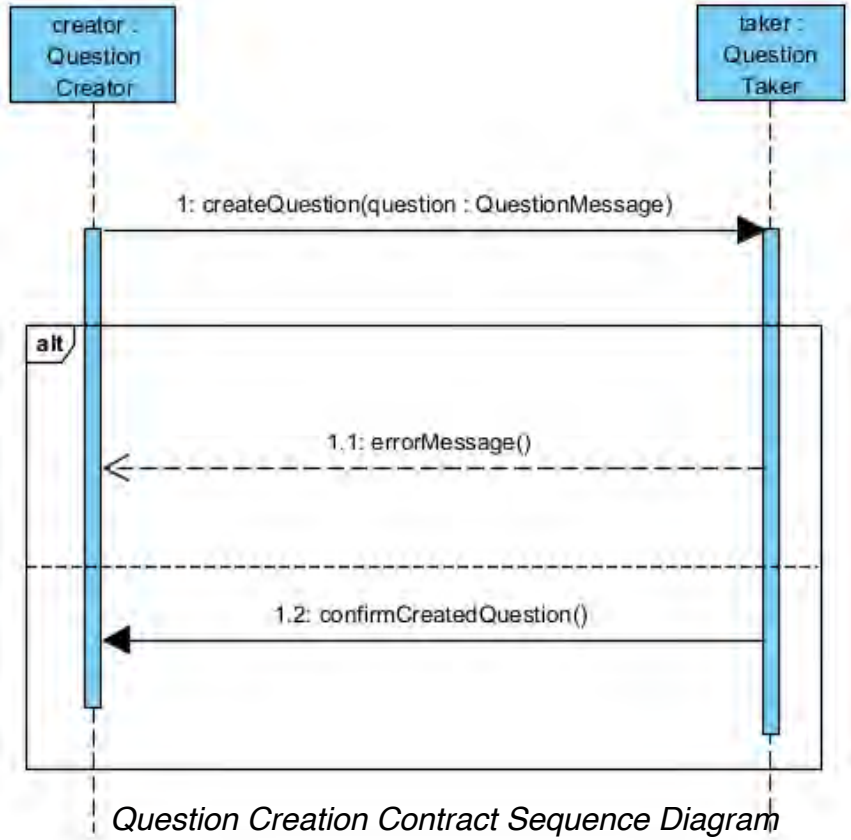
Terms:

1. The creator : Question Creator will implement createQuestion(question : QuestionMessage) by sending an HTTPS POST Request to the taker : Question Taker.
2. The HTTPS Request must have a body that contains a QuestionMessage in JSON format and the members qname : string, qtext : string must exist and not be null, otherwise the creator : Question Creator will receive a validation error.
3. The taker : Question Taker will implement confirmCreatedQuestion() by replying to the HTTPS Request with one of the following status codes:

- 200: if the message is valid.
- 400: if there was a validation error (at least one of the members qname : string, qtext : string was undefined or null).
- 500: if there was an internal server error.

This communication is also documented in the Question Creation Contract Sequence Diagram.

Question Creation Contract Description



Question Creation Contract Sequence Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Question Publishing service.
The service bus : Question Receiver is the Provider and the publisher : Question Publisher is the Consumer.

Terms:

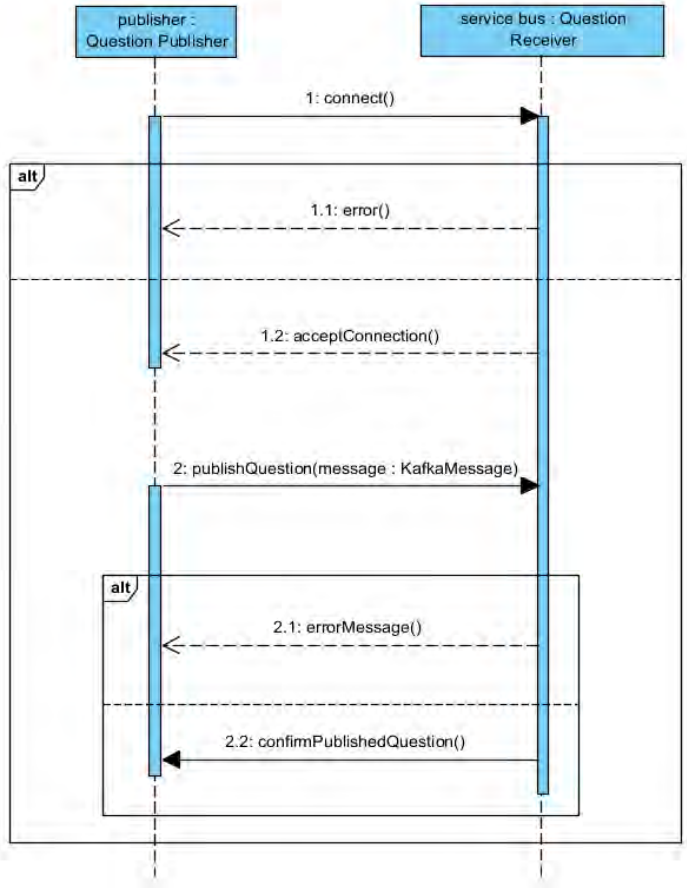
1. The publisher : Question Publisher will send a message to the service bus : Question Receiver. The message that will be sent must be a KafkaMessage that contains a unique key : int and a value : string that contains a QuestionMessage the publisher : Question Publisher has received.
2. In case the service bus : Question Receiver doesn't receive the message, an error is logged.

Conditions:

1. The publisher : Question Publisher must be connected to the service bus : Question Receiver which is a Kafka broker and send a message to the topic "**QUESTION**".

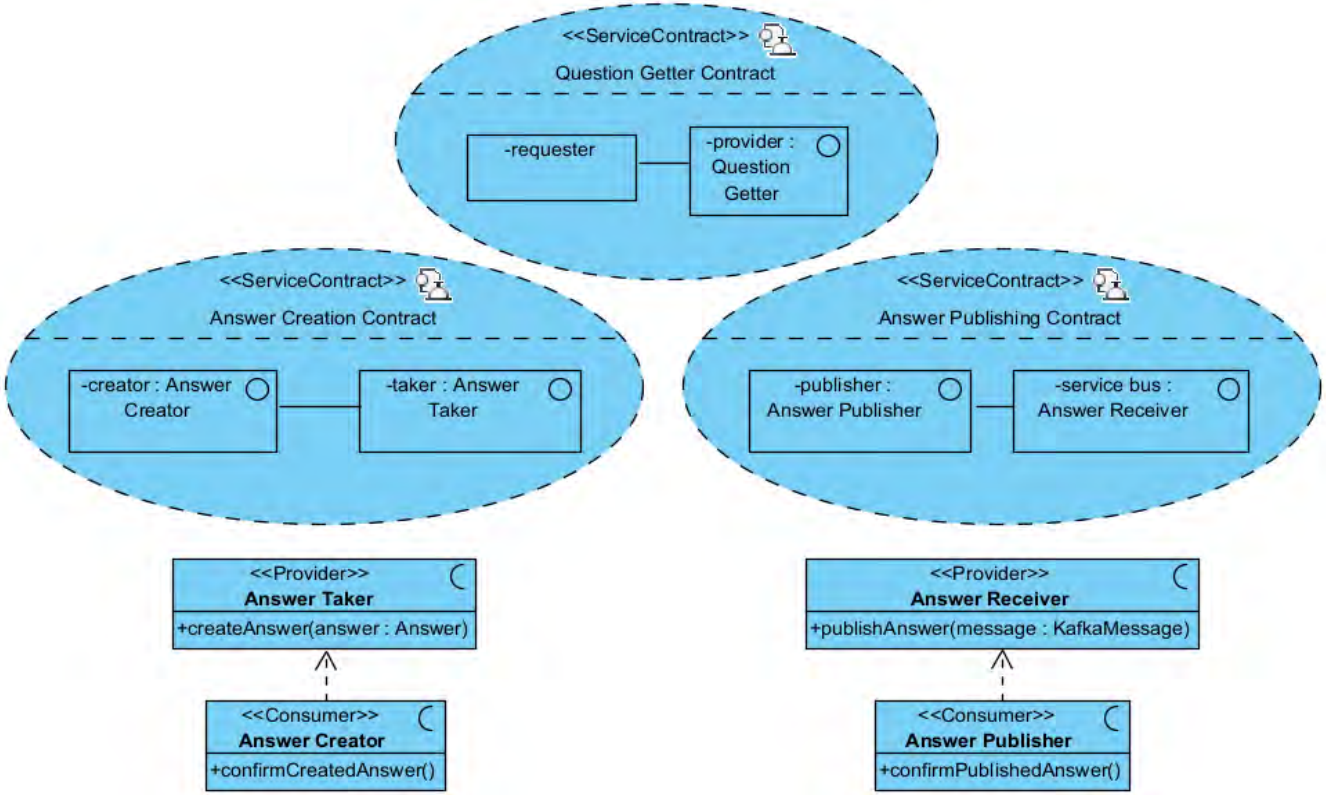
This communication is also documented in the Question Publishing Contract Sequence Diagram.

Question Publishing Contract Description



Question Publishing Contract Sequence Diagram

Service Contract Diagrams



Answers Service Contract Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Answer Creation service.
The taker : Answer Taker is the Provider and the creator : Answer Creator is the Consumer.

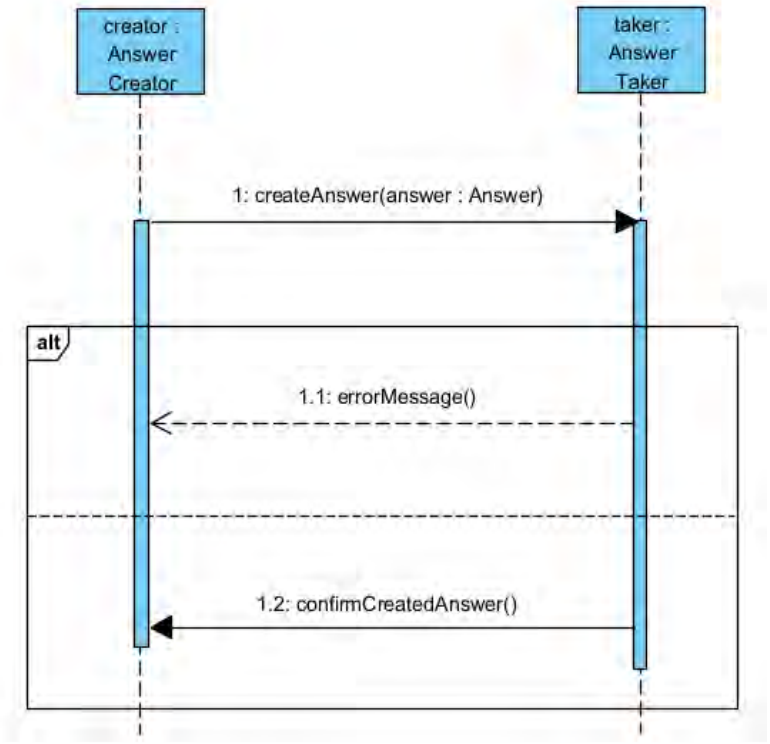
Terms:

1. The creator : Answer Creator will implement createAnswer(answer : Answer) by sending an HTTPS POST Request to the taker : Answer Taker.
2. The HTTPS Request must implement an Answer by having a body that contains an answer : string in JSON format and the parameters containing a member "**id**" that is used as a questionId : int. Both must exist and not be null, otherwise the creator : Answer Creator will receive a validation error.
3. The taker : Answer Taker will implement confirmCreatedAnswer() by replying to the HTTPS Request with one of the following status codes:

- 200: if the message is valid.
- 400: if there was a validation error (at least one of the members questionId : int, answer : string was undefined or null).
- 500: if there was an internal server error.

This communication is also documented in the Answer Creation Contract Sequence Diagram.

Answer Creation Contract Description



Answer Creation Contract Sequence Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Answer Publishing service.
The Answer Receiver is the Provider and the Answer Publisher is the Consumer.

Terms:

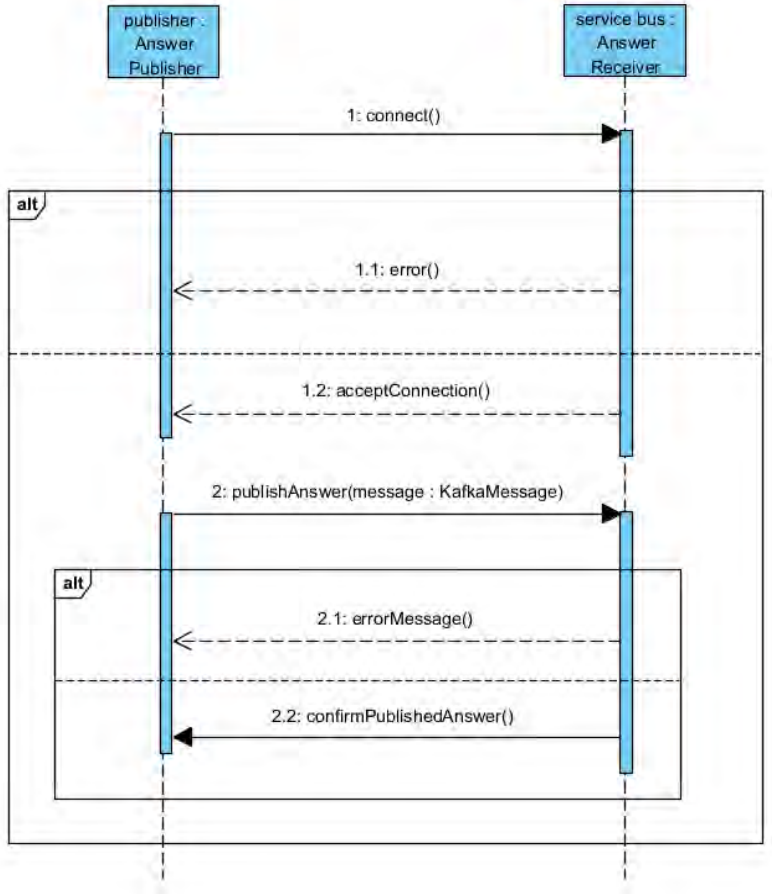
1. The publisher : Answer Publisher will send a message to the service bus : Answer Receiver. The message that will be sent must be a AnswerMessage that contains a key : int and a value : string that contains an Answer the publisher : Answer Publisher has received.
2. In case the service bus : Answer Receiver doesn't receive the message, an error is logged.

Conditions:

1. The publisher : Answer Publisher must be connect to the service bus : Answer Receiver which is a Kafka broker and send a message to the topic "**ANSWER**".

This communication is also documented in the Answer Publishing Contract Sequence Diagram.

Answer Publishing Contract Description



Answer Publishing Contract Sequence Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Question Getter interface.

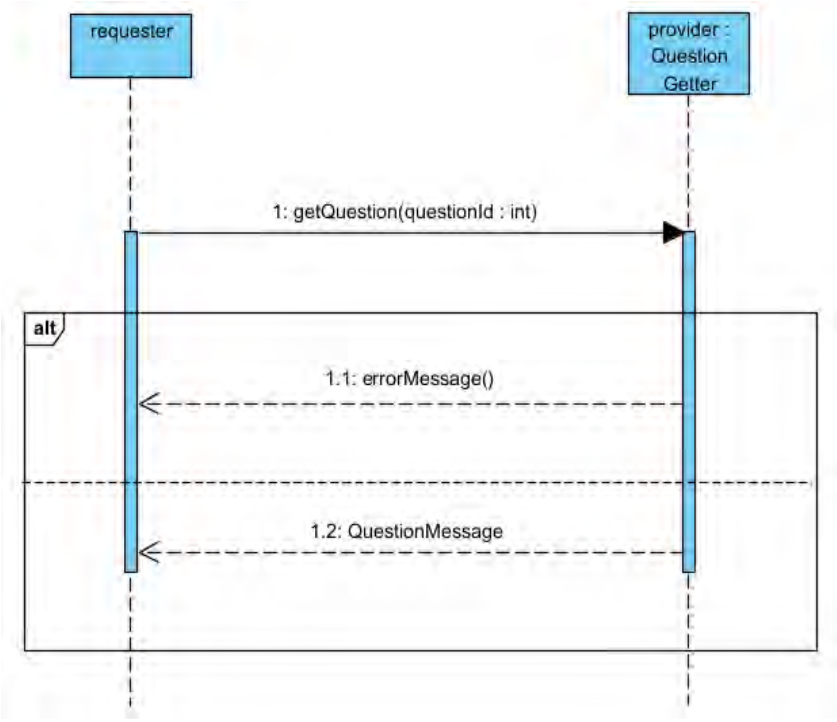
Terms:

1. The requester will implement a getQuestion(questionId : int) by sending an HTTPS GET Request to the provider : Question Getter.
2. The HTTPS Request must contain a member "**id**" in its parameters that is of type **int** and is used as a questionId.
3. The provider : Question Getter will reply to the HTTPS Request with one of the following status codes:

- 200: if the message is valid, and will also return the question (QuestionMessage) that was requested.
- 404: if the question doesn't exist.
- 500: if there was an internal server error.

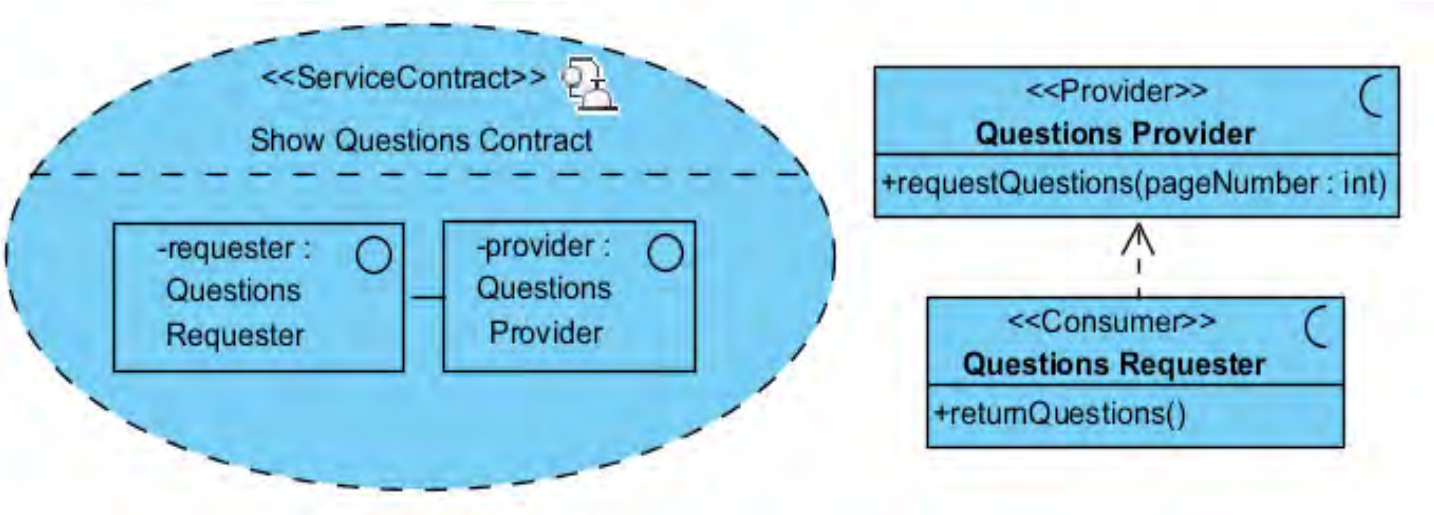
This communication is also documented in the Question Getter Contract Sequence Diagram.

Question Getter Contract Description



Question Getter Contract Sequence Diagram

Service Contract Diagrams



Browse Questions Service Contract Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Show Questions service.
The provider : Questions Provider is the Provider and the requester : Questions Requester is the Consumer.

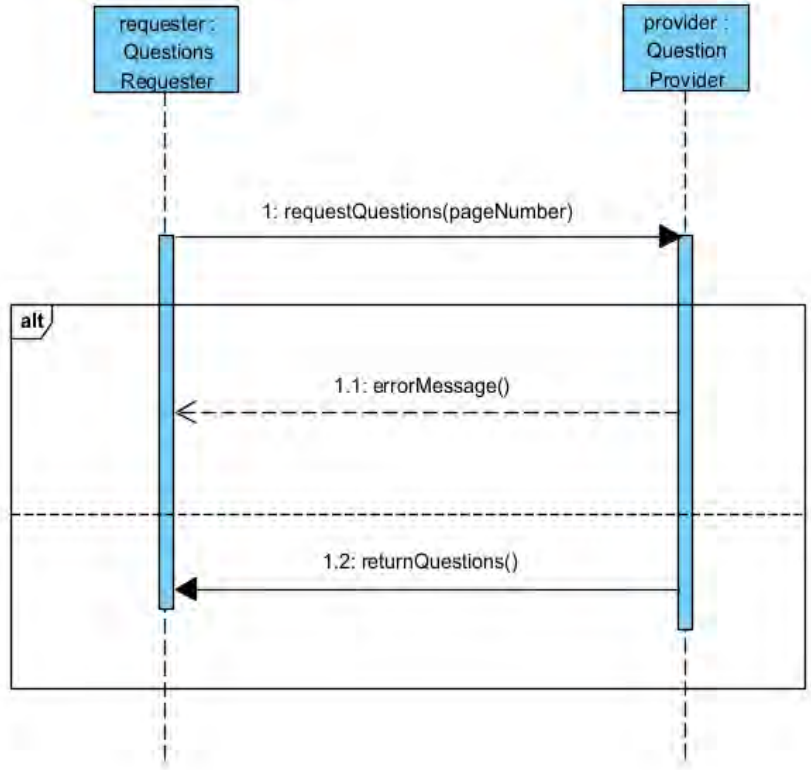
Terms:

1. The requester : Questions Requester will implement requestQuestions(pageNumber : int) by sending an HTTPS POST Request to the provider : Questions Provider.
2. The HTTPS Request must have a body that contains an integer "pageNumber" in JSON format.
3. The provider : Questions Provider will implement returnQuestions() by replying to the HTTPS Request with one of the following status codes:

- 200: if the message is valid and will also return all the questions (list of QuestionMessage) that exist in the page that was requested.
- 404: if the page doesn't exist (invalid pageNumber).
- 500: if there was an internal server error.

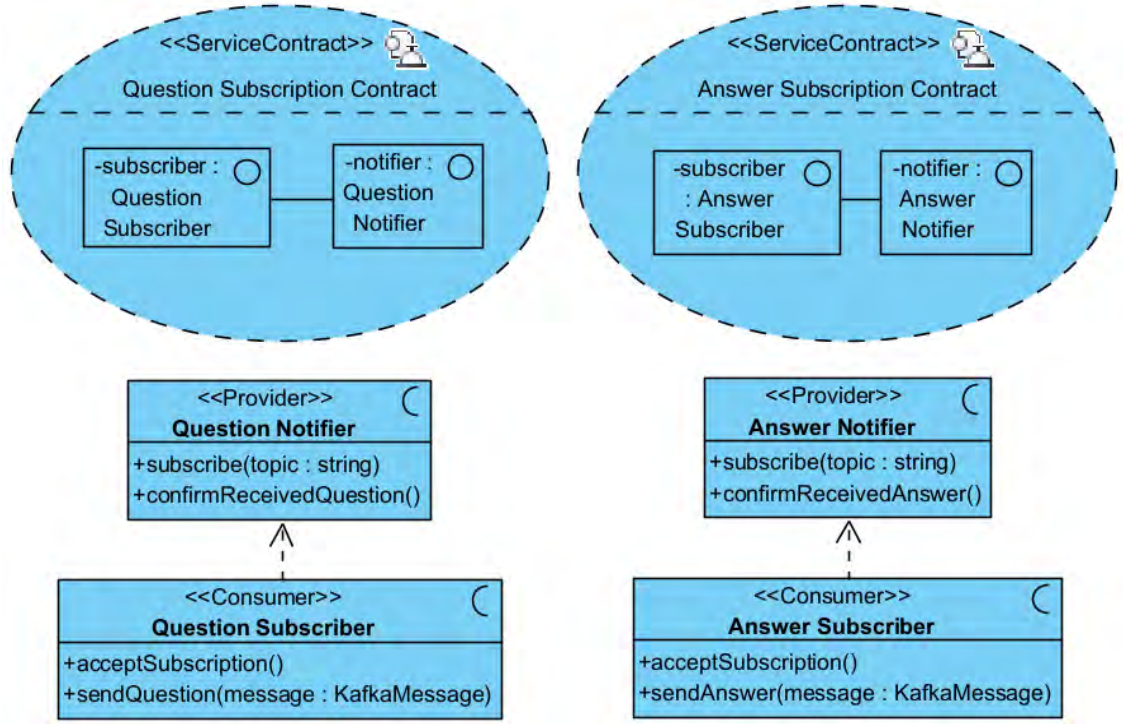
This communication is also documented in Show Questions Contract Sequence Diagram.

Show Questions Contract Description



Show Questions Contract Sequence Diagram

Service Contract Diagrams



Subscription Service Contract Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Question Subscription service.
The Question Notifier is the Provider and the Question Subscriber is the Consumer.

Terms:

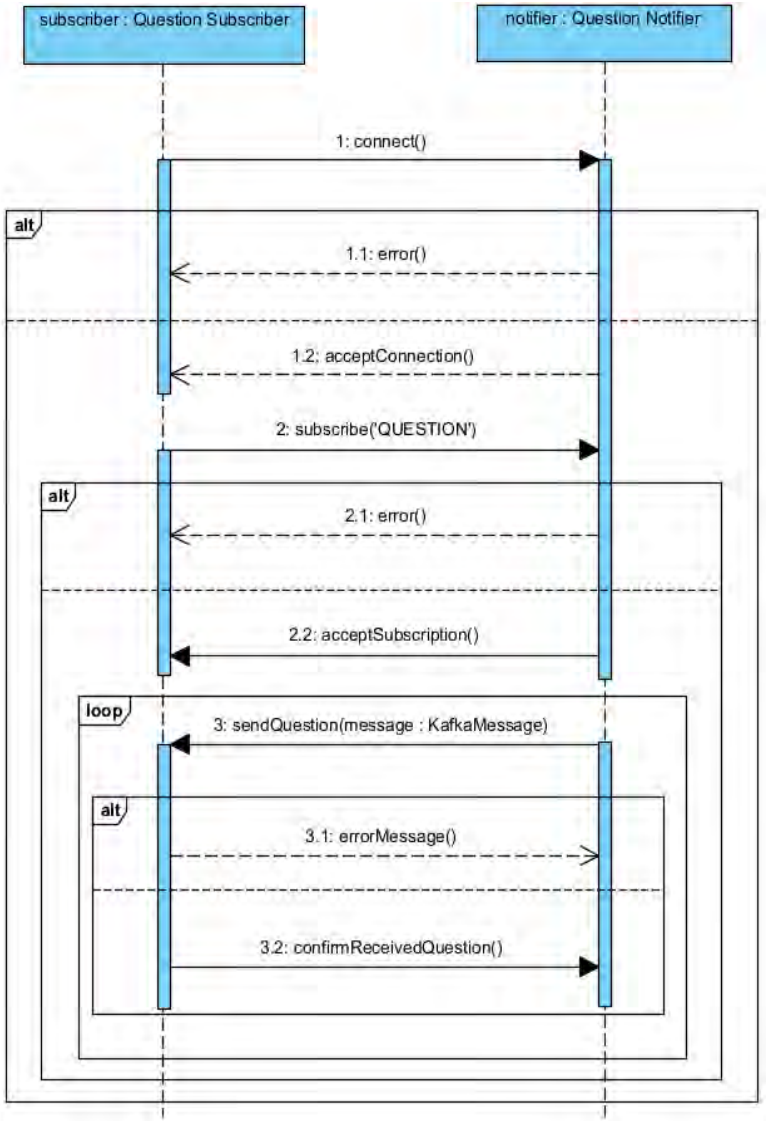
1. The subscriber : Question Subscriber will subscribe to the topic '**QUESTION**'.
2. The subscriber : Question Subscriber receives any new KafkaMessage that is published. That message will contain a QuestionMessage in value : string.

Conditions:

1. The subscriber : Question Subscriber must be connected to the notifier : Question Notifier which is a Kafka broker.

This communication is also documented in the Question Subscription Contract Sequence Diagram.

Question Subscription Contract Description



Question Subscription Contract Sequence Diagram

Service Contract Diagrams

Description:

This contract is created for and used in conjunction with the Answer Subscription service.
The Answer Notifier is the Provider and the Answer Subscriber is the Consumer.

Terms:

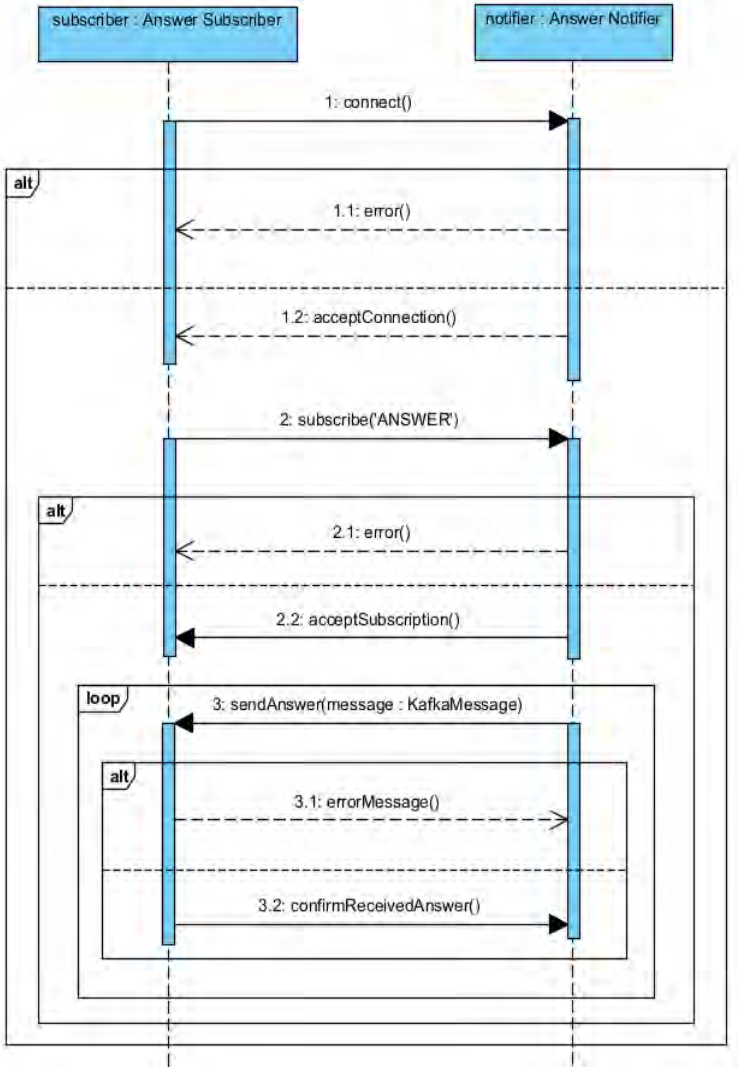
1. The subscriber : Answer Subscriber will subscribe to the topic '**ANSWER**'.
2. The subscriber : Answer Subscriber receives any new KafkaMessage that is published. That message will contain a AnswerMessage in value : string.

Conditions:

1. The subscriber : Answer Subscriber must be connected to the notifier : Answer Notifier which is a Kafka broker.

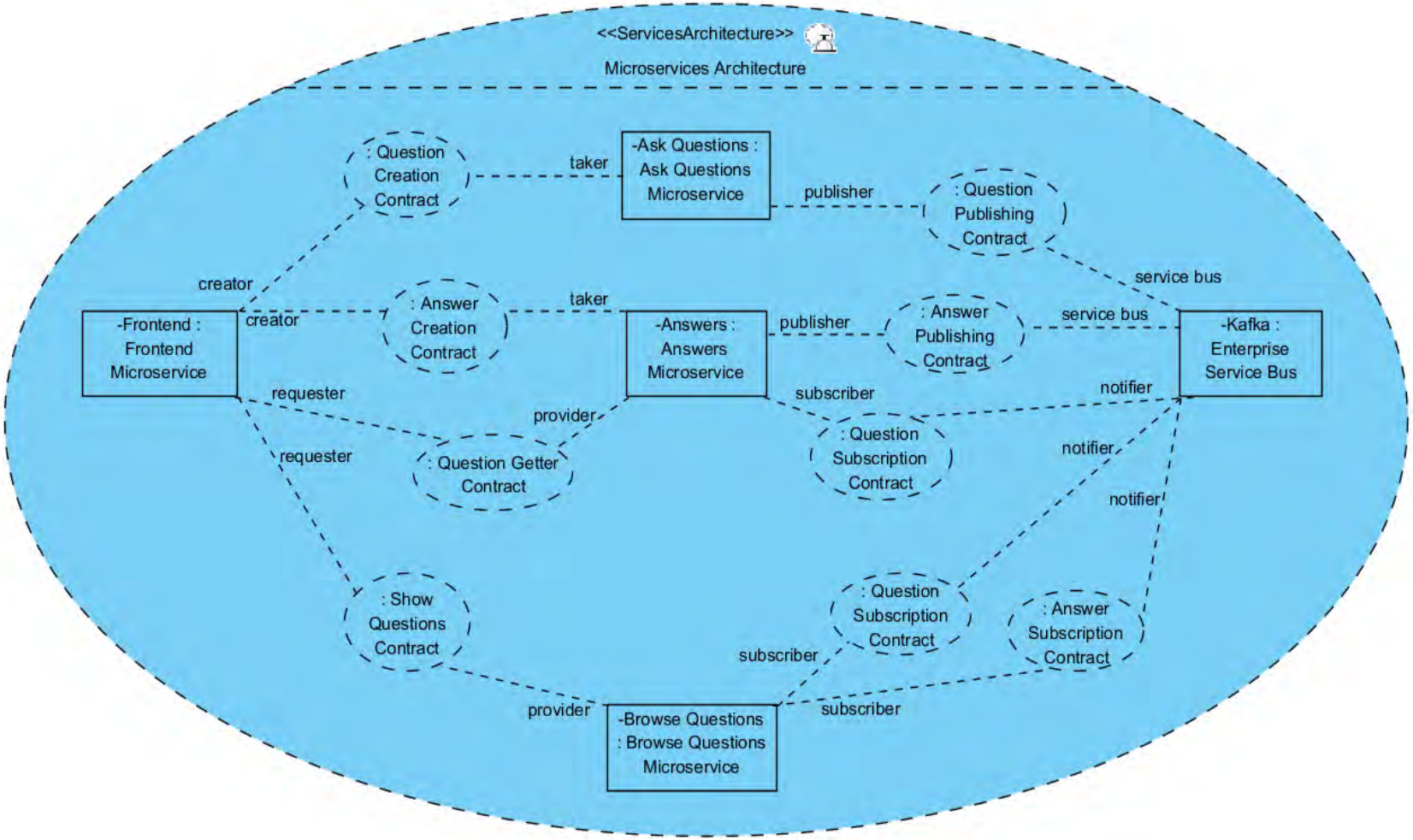
This communication is also documented in the Answer Subscription Contract Sequence Diagram.

Answer Subscription Contract Description



Answer Subscription Contract Sequence Diagram

Services Architecture Diagram



Services Architecture Diagram

SOAML further readings

https://www.omg.org/spec/SoaML